

Part 4 Project Logbook

Project #43: Reconfigurable Neural Processing Unit (NPU) for Energy-Efficient AI at the Edge

Kristelle Sampang
ksam836@aucklanduni.ac.nz
ID: 463680577

Table of Contents

WEEK 1.....	4
03/03/25 – 09/03/25:.....	4
<i>Thursday 6th March 2025</i>	4
<i>Time: 2PM</i>	4
WEEK 2.....	5
10/03/25 – 16/03/25:.....	5
<i>Tuesday 11th March 2025</i>	5
<i>Time: All Day</i>	5
<i>Wednesday 12th March 2025</i>	5
<i>Time: 11PM</i>	5
WEEK 3.....	7
17/03/25 – 23/03/25:.....	7
<i>Tuesday 18th March 2025 – Wednesday 19th March 2025</i>	7
<i>Time: All Day</i>	7
<i>Thursday 20th March 2025</i>	10
<i>Time: 4pm-6pm</i>	10
WEEK 4.....	12
24/03/25 – 30/03/25:.....	12
<i>Monday 24th March 2025</i>	12
<i>Time: 1PM</i>	12
WEEK 5.....	13
31/03/25 – 06/04/25:.....	13
<i>Monday 31st March – Tuesday 1st April</i>	13
<i>Time: All Day</i>	13
WEEK 8.....	14
05/05/25 – 11/05/25:.....	14
<i>Wednesday 7th May</i>	14
<i>Time: All Day</i>	14
WEEK 9.....	15
12/05/25 – 18/05/25:.....	15
<i>Tuesday 13th May – Wednesday 14th May</i>	15
<i>Time: All Day</i>	15
WEEK 10.....	18
19/05/25 – 25/05/25:.....	18
<i>Wednesday 21st May – Thursday 22nd May</i>	18
<i>Time: All Day</i>	18
INTER-SEMESTER BREAK	20
11/07/25 – 20/07/25:.....	20

Friday 11th July 2025 – Saturday 12th July	20
Time: All Day	20
Monday 14th July 2025	22
Time: All Day	22
Tuesday 15th July 2025	23
Time: All Day	23
Wednesday 16th July 2025	29
Time: All Day	29
SEMESTER 2 WEEK 1	32
21/07/25 – 27/07/25:	32
Monday 21st July – Wednesday 23rd July 2025	32
Time: All Day	32
WEEK 3.....	35
04/08/25 – 10/08/25:	35
Monday 4th August 2025	35
Time: All DAY	35
WEEK 5.....	36
18/08/25 – 24/08/25:	36
Monday 18th August 2025	36
Time: All Day	36
MID-SEMESTER	37
01/09/25 – 14/09/25:	37
Friday 12th September 2025	37
Time: All Day	37
WEEK 7.....	38
15/09/25 – 21/09/25	38
Thursday 18th September 2025	38
Time: All Day	38
WEEK 8.....	39
22/09/25 – 28/09/25:	39
Monday 22nd September 2025	39
Time: All Day	39
Wednesday 24th September 2025	42
Time: All Day	42
Thursday 25th September 2025	43
Time: All Day	43
Friday 26th September 2025	43
Time: 1PM	43
Saturday 27th September 2025	44
Time: All Day	44
Complete To-Do list above	44
WEEK 9.....	45
29/09/25 – 05/10/25:	45
Monday 29th September 2025	45
Time: 4PM	45
WEEK 10.....	46
06/10/25 – 12/10/25:	46
Monday 6th September 2025	46
Time: 2PM	46
Wednesday 8th September 2025	47
Time: ALL DAY	47
Thursday 9th September 2025	47
Time: ALL DAY	47

<i>Saturday 11th to Sunday 12th October</i>	<i>49</i>
<i>Time: All Day.....</i>	<i>49</i>
WEEK X.....	51
XX/XX/25 – XX/XX/25:.....	51
XX X XXX 2025.....	51
<i>Time: XX.....</i>	<i>51</i>
XX X XXX 2025.....	51
<i>Time: XX.....</i>	<i>51</i>

Week 1

03/03/25 – 09/03/25:

Thursday 6th March 2025

Time: 2PM

Initial Meeting with Supervisors

Objective: Meet with the supervisors to set weekly meetings and get advice on how to start the research project.

- Set weekly meetings to every Thursday 11.15pm for the first half of Semester One.
- Aim to implement NPU onto an FPGA.
- Gaining sufficient background knowledge will make implementation easier. Literature Review will help with this.
- Utilise UoA library for academic papers, can also find some online but start off with UoA library.
- Attempt to find academic papers individually. Read the abstract and table of contents to check its relevance to the project. Check the paper's sources too for further explanation.
- Make sure to backup the work one – could use online sources like Git but Morteza suggests a physical hard drive.

Week 2

10/03/25 – 16/03/25:

Tuesday 11th March 2025

Time: All Day

Management System

Objective: Create a management system for P4P and start reading the information on Canva

- Created a Sharepoint website through Teams and shared with supervisors.
- Set up a Discord server for communication.
- Created a GitHub repository to backup code files.
- Read the P4P handbook for more context.
- Started to go through 'Finding Information' under Literature Review

Wednesday 12th March 2025

Time: 11PM

Finding relevant papers

Objective: Start to look for academic papers that are relevant to project

Brackets ()

Uses brackets or parenthesis () to build a search using multiple Boolean operators.

Phrase " "

Uses " " to search for a phrase (a term that requires more than one word in the same order. e.g., black hole vs "black hole". These will bring two different search results.

Truncation *

Commonly uses the * symbol. It's like a shortcut to find different forms of a word!
For more details, check out the How to Search the Literature (Advanced): Truncation and wildcards guide.

Wildcard # or ?

Commonly uses # or ? to search for word with various spelling. e.g., wom? n will search for women and woman or colo?r will search for colour and colour.

Proximity W/ and N/

W/ (Within): Keywords in a specific order and close together (e.g., "W/5" means within 5 words of each other). N/ (Near): Keywords close together regardless of order (e.g., "N/5" means a maximum of 5 words apart).

NPU relevance:

- A Reconfigurable Multithreaded Accelerator for Recurrent Neural Networks (<https://ieeexplore.ieee.org/document/9415593>)
- EPU: An Energy-Efficient Explainable AI Accelerator With Sparsity-Free Computation and Heat Map Compression/Pruning (<https://ieeexplore.ieee.org/document/10412195>)

FPGA relevance:

- Acceleration of Rotated Object Detection on FPGA (<https://ieeexplore.ieee.org/document/9680680>)
- Hardware conversion of neural networks simulation models for neural processing accelerator implemented as FPGA-based SoC (<https://ieeexplore.ieee.org/document/6927383>)

Edge Device relevance:

- Acceleration of Rotated Object Detection on FPGA (<https://ieeexplore.ieee.org/document/9803862>)
- Efficient Real-time Object Detection on Edge NPU Device with TOPST AI (<https://ieeexplore.ieee.org/document/10773198>)

Energy-Efficient relevance:

- AI-Optimized Hardware Design for Internet of Things (IoT) Devices (<https://ieeexplore.ieee.org/document/10578352>)

Directly Relevant:

- Energy-Efficient Computing Acceleration of Unmanned Aerial Vehicles Based on a CPU/FPGA/NPU Heterogeneous System (<https://ieeexplore.ieee.org/document/10525057>)
- Custom Hardware Inference Accelerator for TensorFlow Lite for Microcontrollers (<https://ieeexplore.ieee.org/document/9825651>)

Questions:

- There is a lot of knowledge gaps with NPUs. How can I find academic papers that will explain this in a high-level? Or should I just Google?
- There were many types of neural networks that papers talked about, how much impact would this have on the relevance of the paper? Or can we test it using any NN?
-

Search words used:

- Hardware accelerator npu
- Npu edge device
- Npu fpga

Tuesday 18th March 2025 – Wednesday 19th March 2025**Time: All Day**

Title: Back to Basics

Objective: Gain background knowledge on needed for the project

Machine Learning (copied from OneNotes)[But what is a neural network? | Deep learning chapter 1](#)

Notes:

- What are neural networks?

Example: recognising a number between 0-9 from a 28x28 pixel image greyscale.

Input layer: Each pixel in the image will have a value between 0 to 1 that corresponds to how bright the pixel is. This is the first layer of the network.

Hidden layers: From the input layer to a hidden layer, relevant data is identified and separated, and passed onto the next hidden layers until it reaches the output layer.

Output layer: The final prediction of what it thinks the number is.

- Why are the networks layered?

The numbers can be broken down in many pieces. These pieces have a particular pattern e.g. loop, straight lines. Then, these little patterns will have smaller components. When these smaller components go through the layer, it will light up neurons that correspond to it, and as it passes through the layers, it will detect the bigger pieces until it can determine / make a prediction.

[Deep Learning | What is Deep Learning? | Deep Learning Tutorial For Beginners | 2023 | Simplilearn](#)

Notes:

- Deep learning is inspired by the structure of the human brain using neural networks.
- Deep learning vs machine learning:
 - ML: need to tell the machine the features
 - DL: features are picked out using neural networks without human intervention
- How does neural networks work?
 - Activation function determines if the neuron is activated. The output makes a prediction based on what neuron is turned on.

[Deep Learning Basics: Introduction and Overview](#)

Notes:

- Deep learning is more about representation of patterns
- Data is usually trivial, it's harder to display using algorithms
- Purpose: to remove human intervention
- Skipped to 38:30
- Loss functions: mean squared error, cross entropy loss, gap between prediction and truth
 - Update the weights and biases to decrease loss function
- Rest of the lecture seems like it was going into the detail of the accuracy of ML (does not feel relevant)

Deep Learning in One Slide

- | | |
|---|--|
| <ul style="list-style-type: none">• What is it:
Extract useful patterns from data.• How:
Neural network + optimization• How (Practical):
Python + TensorFlow & friends• Hard Part:
Good Questions + Good Data• Why now:
Data, hardware, community, tools, investment• Where do we stand?
Most big questions of intelligence have not been answered nor properly formulated | <ul style="list-style-type: none">Exciting progress:<ul style="list-style-type: none">• Face recognition• Image classification• Speech recognition• Text-to-speech generation• Handwriting transcription• Machine translation• Medical diagnosis• Cars: drivable area, lane keeping• Digital assistants• Ads, search, social recommendations• Game playing with deep RL |
|---|--|

<http://neuralnetworksanddeeplearning.com/chap1>

Notes:

- Did not finish the entire chapter – went into detail which I felt like is not necessary background knowledge for this research

CNN

[What are Convolutional Neural Networks \(CNNs\)?](#)

Notes:

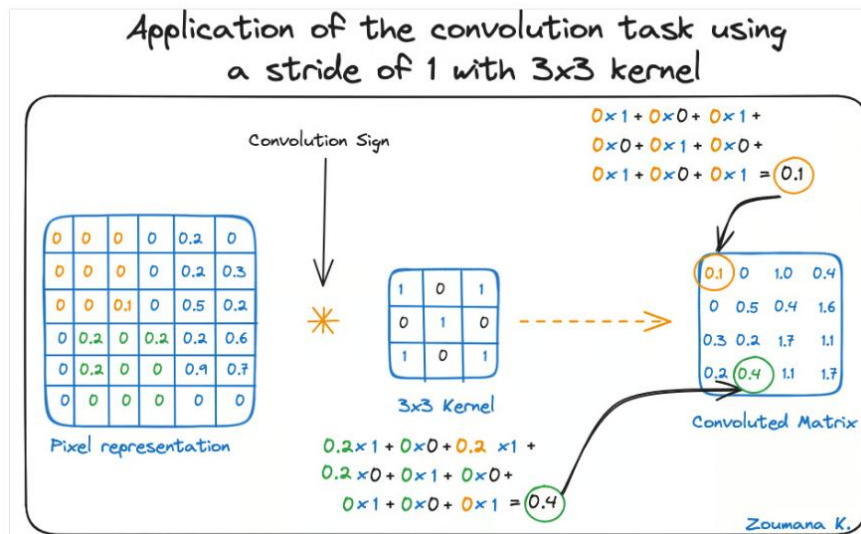
- Filters in CNN perform the pattern recognition
- A filter is a 3x3 block that specifies a pattern to look for e.g. straight line or an L shape
 - This block is compared to a 3x3 section of the given image and checks if it is the same. It continues to slide over the image until all 3x3 have been mapped on the image.
- Pooling: combining all the numeric arrays from each filter
- The filters become more abstract the deeper it goes into the neural network where it recognises the larger part of a pattern

<https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns>

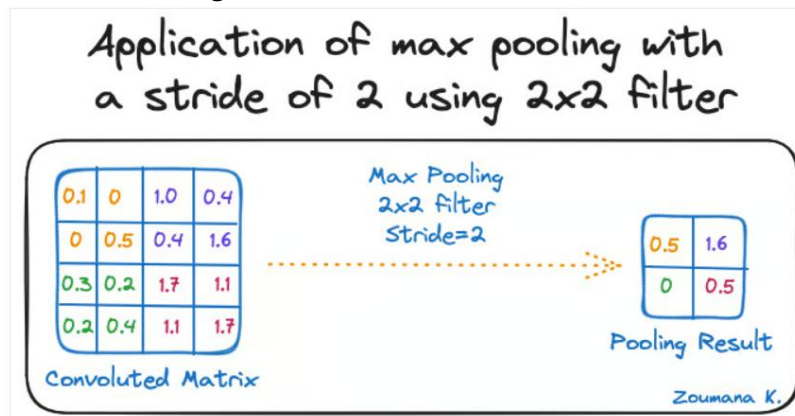
Notes: (only read key concepts)

- Used for object recognition, image classification, detection, and segmentation.
- Can identify and extract patterns and features
- Made up of 4 main layers: convolutional layers, rectified linear unit, pooling layers, and fully connected layers
- Convolutional layer:
 - Filter: Small matrices of the same size contain a pattern that is part of the image. These matrices are compared to the entire image.

The dimension of the convoluted matrix depends on the size of the sliding window. The higher the sliding window, the smaller the dimension.



- Rectified Linear Unit (ReLU) activation function: helps learn the relationships between the features of the image
- Pooling layer:
 - Pulls the most significant features from the convoluted matrix



- Fully connected layers:
 - Output layer where all extracted features are transformed into predictions

DNN

<https://www.datacamp.com/tutorial/introduction-to-deep-neural-networks>

Notes:

- A Neural Network is deep when it has more than one hidden layer

RNN

[The Power of Recurrent Neural Networks \(RNN\)](#)

Notes:

- Handles sequence data
- Has loops so can use information from past (memory)
- A recurrent neuron has its own loop, called the hidden state
- Output depends on the hidden state
- Hidden state is calculated based on the current input and the previous hidden state
- Hidden state is updated at every step allowing to learn based on previous hidden states
- During updates, the weights of each neuron also changes. If the weight is too small, then it can't really use the previous state. If too large then it becomes unstable

<https://www.ibm.com/think/topics/recurrent-neural-networks>

Notes:

- Has a specific sequence and uses this information for a prediction
- Hidden states acts like a memory that stores information about previous inputs
- Same weight within each layer of the network
- Sigmoid function: determines whether data should be kept or removed

Learning Outcomes:

- How a neural network works and its main three layers: input, hidden, and output layer
- Deep learning is based on learning patterns and requires no human intervention
- Neural networks mimic how the brain works and is trained using large batches of images
- DNN becomes “deep” because it has more than one hidden layer
- CNN uses a filter that slides around the entire images and uses that to compare
- RNN has a memory that updates as it learns through its hidden state loops

Thursday 20th March 2025

Time: 4pm-6pm

NPU focused background research

Objective: Gain more background knowledge about NPUs

NPU (copied from OneNote):

WHAT IS NPU | MOST ADVANCED AI COMPUTER PROCESSOR

Notes:

- Specialised processor to accelerate AI and ML workloads
- Tailored for complex computation
- Integrated within the main CPU but it can be discrete in some cases like data centres
- NPUs are optimised for heavy computations in deep learning like facial recognition and autonomous driving
- Avoids having to send to cloud for processing so its better privacy

The NEW Chip Inside Your Phone! (NPUs)

Notes:

- Cloud hardware might be more powerful but the latency / delay is the issue
- Prefer to run AI functions locally

<https://www.ibm.com/think/topics/neural-processing-unit>

Notes:

Advantages:

- Parallel processor so many multiple neural network operations run concurrently

- Low precision arithmetic (usually 8 bits or lower) so computational complexity is less (cause there is less data stored) so it's more energy efficient (check <https://research.ibm.com/blog/low-precision-computing>)
- High-bandwidth memory: can store and access memory quicker for tasks

How does it work?

- Purpose built for problem-solving functions that can improve overtime using different types of data and inputs
- NOT built for precise computations so that it can prioritise speed and efficiency for AI
- Has addition and multiplication modules used for AI tasks that calculate things like matrix multiplication/additinoal, convolution, and dot product etc
- Normal CPUs might need thousands instructions to complete one neuron processing but NPU can do similar operation in one
- Uses synaptic weights (how strong the connection is between two nodes) for storage and computation for more efficiency

Monday 24th March 2025**Time: 1PM**

Energy-Efficiency and Systolic Arrays Background Knowledge

Objective: Gain background knowledge

Energy-Efficiency (copied from OneNote):<https://research.ibm.com/blog/low-precision-computing> Notes:

- Because there is so many instructions in an AI calculation, each instruction does not need high precision and there is a lot of redundant precision that can be excluded
- Can use some form of estimation without sacrificing accuracy
- 16 and 98 bit precision is industry accepted
- Full-precision computing: Model weights are larger so more storage needed
- With "smaller" computations like $a*b$, the answer will always be the same regardless if its 32 or 64 bits
- In a neural network, if one neuron makes a mistakes then the other neuron can capture the signal
 - Meaning: the neural network will be given the one big task. Each neuron will do little bits of the task. If one neuron messes up their tasks, then the other neurons will still be accurate enough to give a reasonable prediction

<https://www.reflexces.com/newsroom/6-reasons-to-use-fpgas-for-ai#:~:text=Energy%20efficiency,-One%20of%20the&text=AI%20tasks%2C%20such%20as%20machine,consumption%20than%20CPUs%20and%20GPUs.>

Notes:

- FPGA can become energy efficiency if their configurations are optimised for low power consumption

<https://www.ibm.com/think/topics/fpga-vs-gpu>

Notes:

- Don't need much more background knowledge about energy efficiency. Can just see this through performance testing during implementation

Week 5**31/03/25 – 06/04/25:**

Monday 31st March – Tuesday 1st April**Time: All Day**

Literature Review

Objective: Read at least 10 papers and start on the literature review

- Created a mindmap with references noted -> makes literature review easier to write

Week 8**05/05/25 – 11/05/25:**

Wednesday 7th May**Time: All Day**

Literature Review Feedback

Objective: Receive Feedback on the literature review and go with a direction

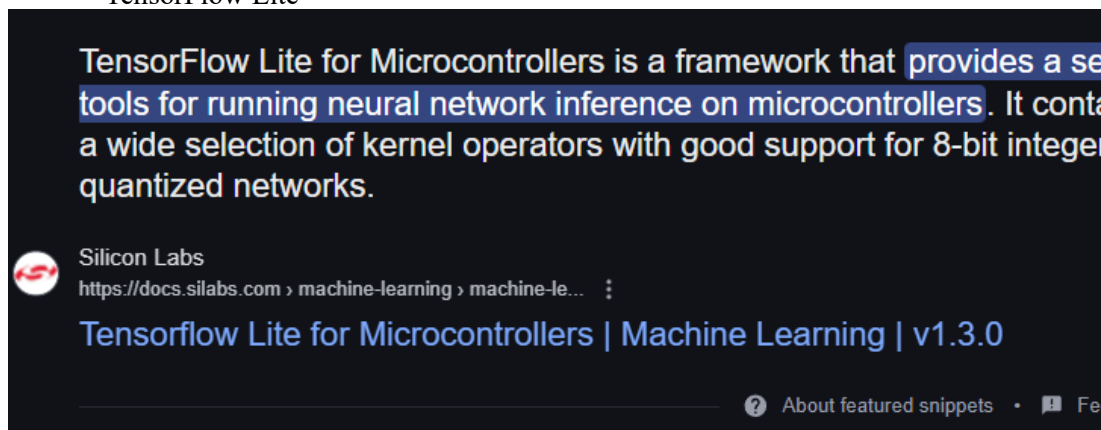
- Fix the formatting
- Some aims and objectives were too ambitious, find something simpler

Tuesday 13th May – Wednesday 14th May**Time: All Day**

Pre-trained Machine Learning Models

Objective: Have a look at the available CNN models available for the FPGA

- DE1-SOC ARM Processor: ARM* Cortex* A9
Paper: A CNN Inference Accelerator on FPGA With Compression and Layer-Chaining Techniques for Style Transfer Applications uses the same
- ImageNet: <https://www.kaggle.com/models/iree/mobilenet-v3>
- There is documentation on how to deploy TinyML in Altera
- ‘Deploying tinyML on Altera FPGA’
 - TFLite is the ML used in the example
 - Instructions for both NIOS and Arm
 - Can run TensorFlow Lite Micro on NIOS II, will need to use ARM if we want TensorFlow Lite



Found a project from Coloumbia University that basically replicates what we’re trying to do:

<https://www.cs.columbia.edu/~sedwards/classes/2025/4840-spring/proposals/Systolic.pdf><https://www.cs.columbia.edu/~sedwards/classes/2025/4840-spring/designs/Systolic.pdf><https://www.cs.columbia.edu/~sedwards/classes/2025/4840-spring/index.html>

Links that might be useful but have not been saved

<https://www.cse.wustl.edu/~roger/560M.f17/01653825.pdf>

How do Systolic Arrays work?

To implement systolic arrays in FPGA, we must understand the fundamental behaviour of a systolic array. Systolic arrays are made up of a network of processing elements to perform simple operations alongside other PEs (JOHN P. CONDORODIS)

So, how does a processing element work?

To put simply, a processing element is just a cell that ‘processes’ something. For our case, we want this to do a multiply and accumulate operation. For the sake of simplicity, I will just use multiply as an example for my understanding.

“The processing element (PE) is essentially an arithmetic logic unit (ALU) with attached working registers and local memory (Briggs and Hwang 1984).”

If we take this quote and apply to our PE, to find the input and outputs, we simply think of “What

do we need to give the ALU so that it performs multiplication?”
 This tells us that we just need input_a and input_b with output_c

TABLE 2

GEOMETRIC CONFIGURATIONS AND CORRESPONDING FUNCTIONS

PROCESSOR ARRAY STRUCTURE	COMPUTATIONAL FUNCTION
linear arrays	Fourier transform, sorts, priority queue
orthogonal arrays	orthogonal matrix arithmetic, graph algorithms involving adjacency matrices
hexagonal arrays	band matrix arithmetic, transitive closure, pattern matching, relational database operations
trees	searching algorithms, parallel function evaluation, recurrence evaluation
triangular arrays	inversion of triangular matrix, formal language recognition

The input of the systolic array is the weight and the activation. “A CNN Inference Accelerator on FPGA With Compression and Layer-Chaining Techniques for Style Transfer Applications” mentions data compression and how they’ve used an 8 bit unsigned input and output

Questions to answer:

- What data type are the weights and activations to the input of the MAC operations?
- If we end up using a Vector Processing Unit Block and a Convolution Processing Unit Block, and the data type of above is floats, can we use a LUT table instead?

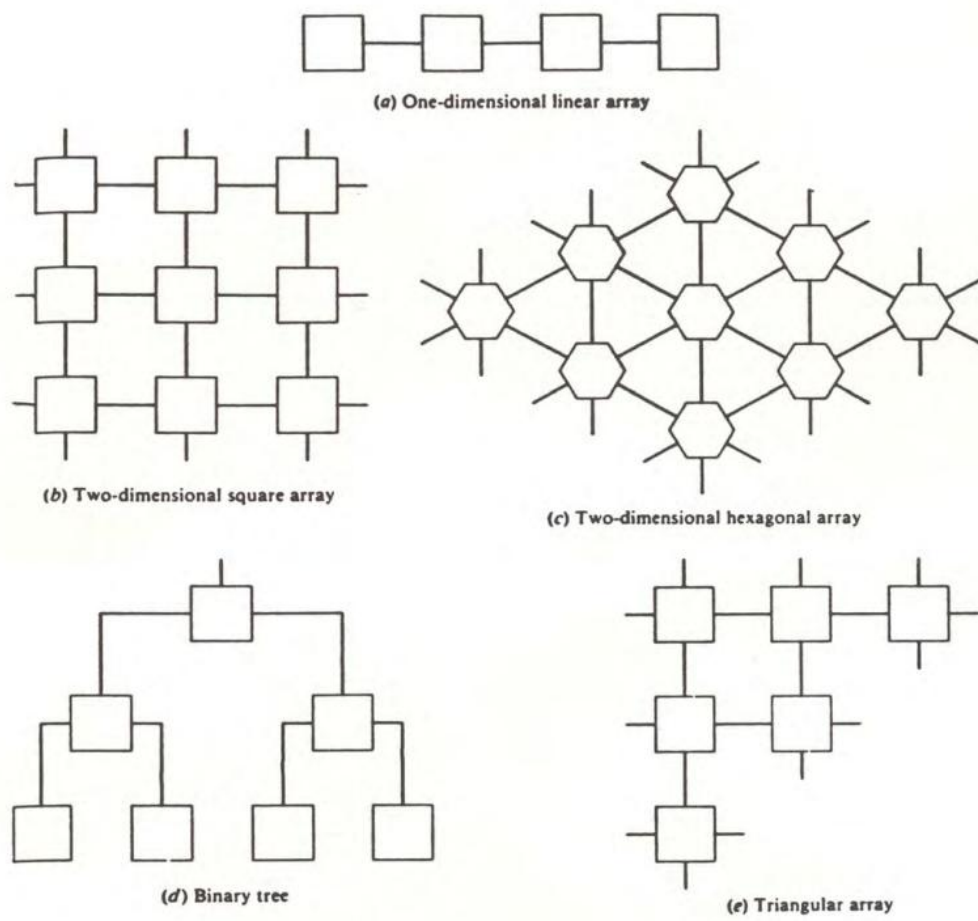


Figure 2. Typical Systolic Architecture Configurations.

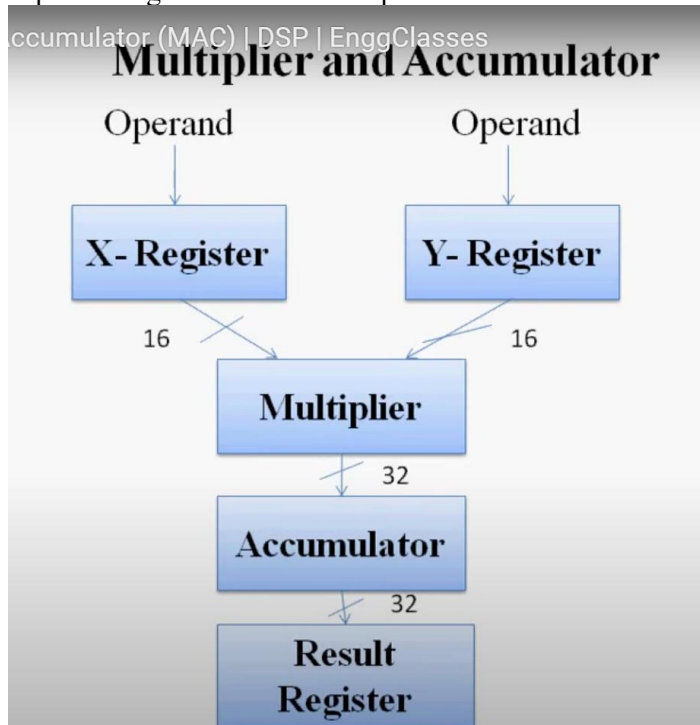
Wednesday 21st May – Thursday 22nd May

Time: All Day

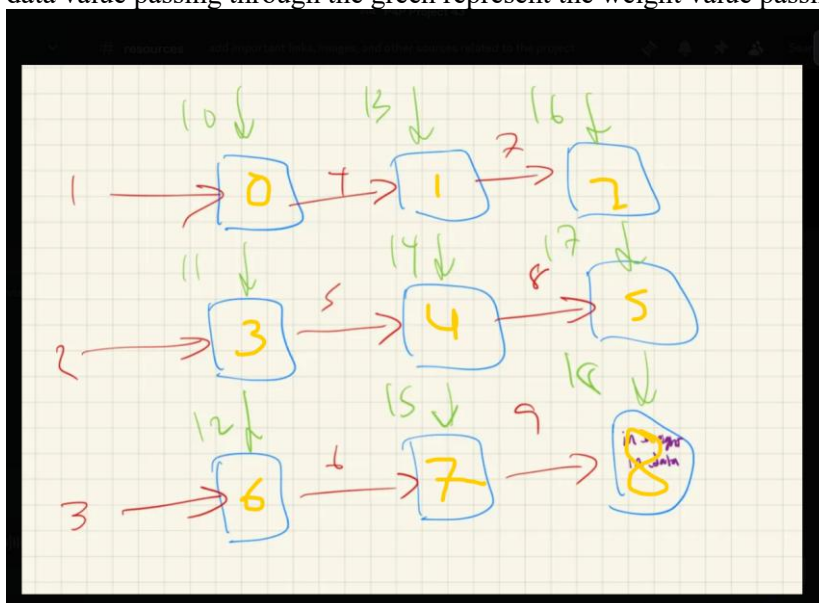
Creating Processing Element and Systolic Array in VHDL

Objective: Create a 3x3 systolic array and test via ModelSim

- A processing element is made up of this:



- Basically, the result and the accumulation stays within the PE. It doesn't actually get passed through the systolic array. The output of the systolic array is the result register
- This is the systolic array architecture that we are trying to implement. Between each PE there is an interconnection signal that needs to be instantiated. The red lines represent the data value passing through the green represent the weight value passing through.



Systolic Array Implementation on VHDL:

- There is something wrong with the processing element where we are struggling to implement the accumulator register
- The systolic array itself is passing through the data properly when we only do multiplication
- The data is staggered through from the top left corner to the bottom right corner
- Manually injecting the data

Ideas to implement it better:

- Use a control unit to pass the data through the entry points of the systolic array
 - Consider the difference between row major vs column major
- Try adding the memory buffer of the PE in the top level of the systolic array

To do:

- Get a proper understanding on how systolic arrays work
- Then move onto the sparse matrices

INTER-SEMESTER BREAK

11/07/25 – 20/07/25:

Friday 11th July 2025 – Saturday 12th July

Time: All Day

Catch up!

Objective: Get back into P4P by revising what has happened previously

What is happeninng? I am lost.

Based on Pratham's modified PEs and Systolic Array:

- In the original code, we had the accumulator value passed throughout the systolic array. THIS IS NOT WHAT WE WANT! The current code has a local accumulator for each PE. (This simplifies the code)
- The systolic array is still hardcoded to feed in the inputs

From these results, what should we do?

- I will be focusing on how to dynamically adapt to the systolic array to any NxN matrix input
- This is the purpose of the control unit
- The idea is to have a set systolic array size, for example 16x16 systolic array that can only take up to 16x16 inputs. If an input is the smaller, then some PEs will "turn off"

How do we do this?

- We need to figure out a way to turn off PEs. I think this is more of a VHDL syntax/code problem than a logic problem.
- Each PE can have an Enable bit to check whether or not it should be processed
- Then on the higher level, systolic array, it needs to know it should stop.
- Another thing we need to consider is to check if we can find a formula for calculating how many clock cycles are needed for a given input.

NxN	Clock Cycles
1	1
2	4
3	7
4	10
5	13
6	16

Based on this table, we know that the formula is: **3n-2**

To get the array size, use the `length attribute

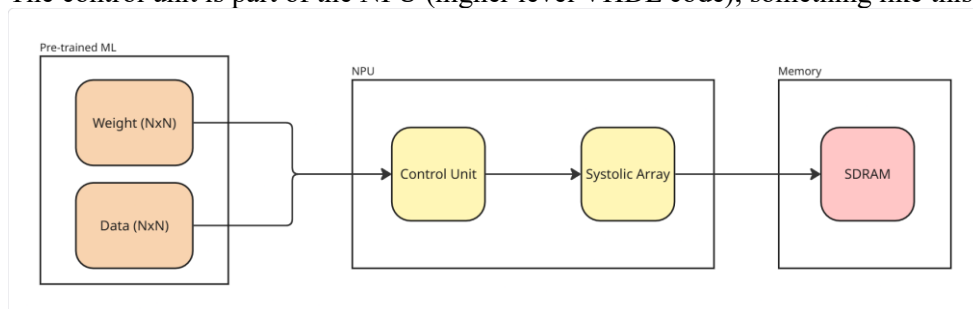
List of VHDL Tick Attributes

Below is a list of VHDL predefined attributes. This list is a subset of all of the VHDL attributes available, but it is a list of the attributes that are used most often. This list is a subset of the list found [here](#).

T'BASE is the base type of the type T.
T'LEFT is the leftmost value of type T. (Largest if downto)
T'RIGHT is the rightmost value of type T. (Smallest if downto)
T'HIGH is the highest value of type T.
T'LOW is the lowest value of type T.
T'IMAGE(X) is a string representation of X that is of type T.
A'LEFT is the leftmost subscript of array A or constrained array type.
A'RIGHT is the rightmost subscript of array A or constrained array type.
A'RANGE is the range A'LEFT to A'RIGHT or A'LEFT downto A'RIGHT .
A'LENGTH is the integer value of the number of elements in array A.
S'EVENT is true if signal S has had an event this simulation cycle.

Okay now that we have this basic information, what will the control unit actually do?

- The control unit is part of the NPU (higher level VHDL code), something like this



- The inputs and outputs of the control unit will be:

Input	Output
Clk (bit 1)	Array Size (int)
Reset (bit 1)	Weight_Input (bit 8 size of however big the systolic array is)
Weight (NxN)	Data_Input (bit 8 size of however big the systolic array is)
Data (NxN)	Injection_Rdy (bit 1)
	PE_On (bit 1 size of however big the systolic array is)

- The idea is for the control unit to analyse the given weight and data matrix. It will get the size of both arrays, assuming they will always be identical (for now). Then, based on this information, it will calculate the clock cycles based on the formula. Then, one by one, it will feed the correct values into the systolic array.
- PE_On is a mask that represents which PE should be turned on

Systolic array:

- How will it turn off the PE? It will take in the mask and align it with the PE enable

Monday 14th July 2025

Time: All Day

Systolic Array with dynamic PE size

Objective: Continue to figure out how to turn off specific PEs for a fixed NxN matrix input

Research Papers:

A Design and Simulation for Dynamically Reconfigurable Systolic Array

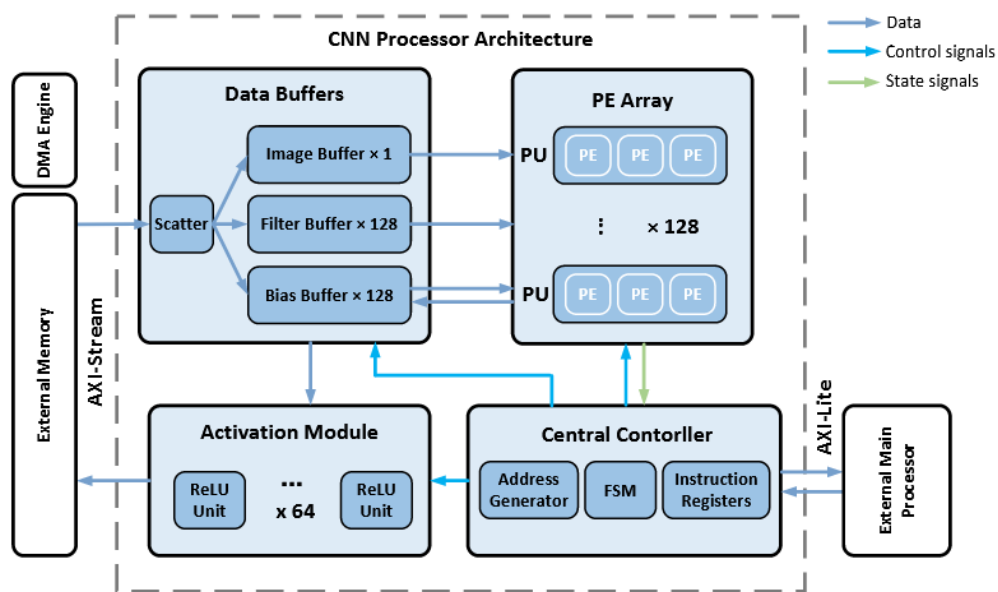
- Paper focuses on simulating reconfigurable systolic array in VHDL
- Reconfigurable Systolic Array Block (RSAB) uses multiplexers to configure different arrangements of 4 PEs
- Not sure if this is applicable to the project

Mapping multiple algorithms into a reconfigurable systolic array

- Multiple algorithm are mapped into a reconfigurable systolic array that can compute one algorithm at a time
- Systolic array can be expanded using time or space redundancy (??) so it can computer multiple algorithms simultaneously
- Hard to follow, I have no idea what this was talking about

An Energy-Efficient Convolutional Neural Network Processor Architecture Based on a Systolic Array

- Cyclic Tiling (??) reduces hardware resource usage
- By reusing the input image data and weight data combined with multi-level storage methods, power of memory access is reduced and high-energy efficiency is obtained.



-
- Looks helpful for future implementation, but not what I need right now

Questions:

- Are we going to stick to one arrangement of a systolic array e.g. orthogonal or something like the diagonally configuration? Or should we allow the PEs to arrange itself in different orders? I think it's better to look into the best arrangement for CNN algorithms and optimise that
- Instead of simply just turning on/off a processing element, can maybe we can somehow use the disabled PEs somehow so that we increase hardware utilisation

Code:

- Processing unit now has an input enable bit and has a conditional statement to ensure that it only processes data if enabled
- “custom_types.vhd” is basically a copy of what we had in recop, to simplify coding for us

Tuesday 15th July 2025

Time: All Day

Create a dynamic systolic array

Objective: Learn how to use the “generate” statement

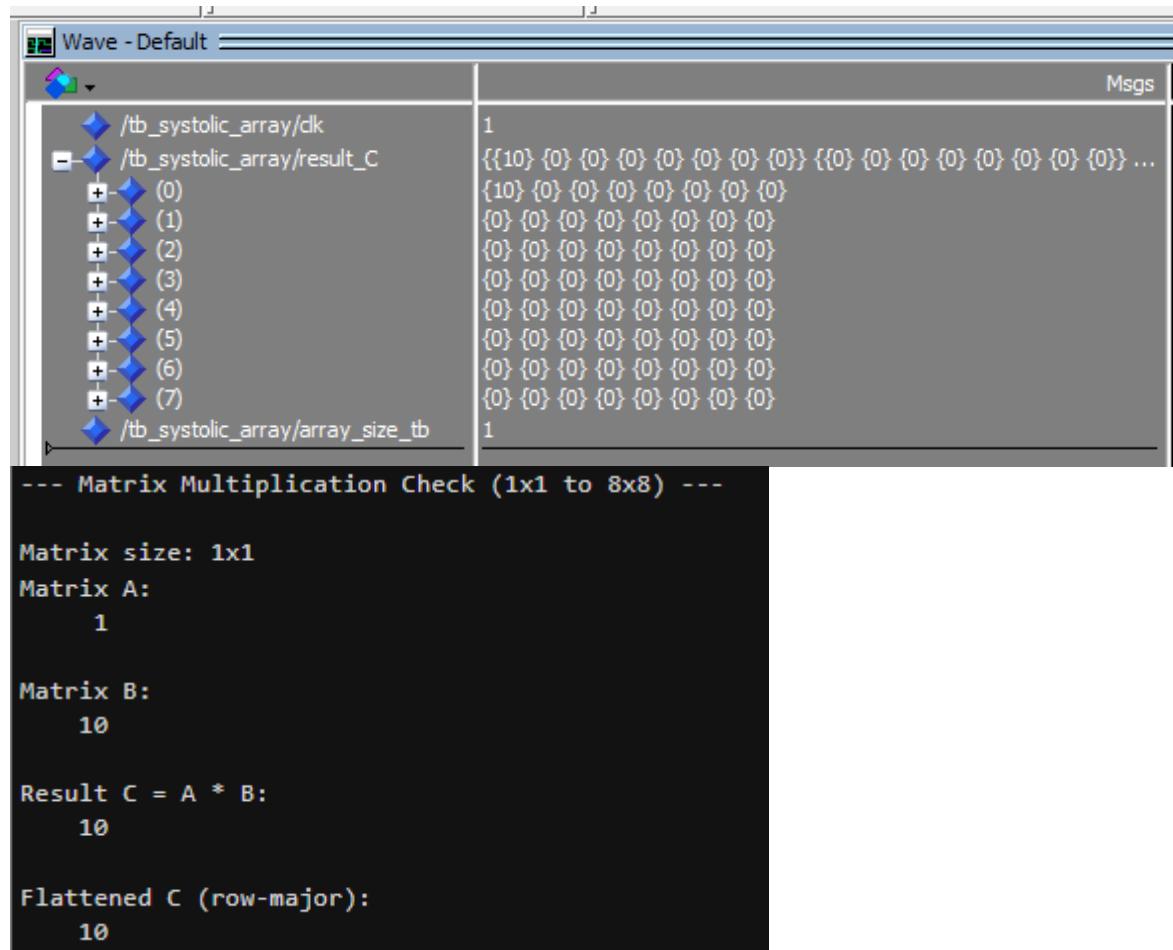
To Do:

- Learn how to use the “generate” statement
- Generate 8x8 PE systolic array
- Test with a 1x1, 2x2,3x3....8x8 array input
- Create a separate file for control unit

Testing Screenshots

Set up: 8x8 Systolic Array. Tests will start from 1x1 all the way to 8x8

1x1



The screenshot shows a logic simulator interface. The top part displays a wave for the signal `/tb_systolic_array/result_C`, which is a 1x1 matrix. The bottom part shows a console output for a 1x1 matrix multiplication test.

Wave - Default

Signal	Value
<code>/tb_systolic_array/dk</code>	1
<code>/tb_systolic_array/result_C</code>	{{10} {0} {0} {0} {0} {0} {0} {0}} {{0} {0} {0} {0} {0} {0} {0} {0}} ...
<code>(0)</code>	{10} {0} {0} {0} {0} {0} {0} {0}
<code>(1)</code>	{0} {0} {0} {0} {0} {0} {0} {0}
<code>(2)</code>	{0} {0} {0} {0} {0} {0} {0} {0}
<code>(3)</code>	{0} {0} {0} {0} {0} {0} {0} {0}
<code>(4)</code>	{0} {0} {0} {0} {0} {0} {0} {0}
<code>(5)</code>	{0} {0} {0} {0} {0} {0} {0} {0}
<code>(6)</code>	{0} {0} {0} {0} {0} {0} {0} {0}
<code>(7)</code>	{0} {0} {0} {0} {0} {0} {0} {0}
<code>/tb_systolic_array/array_size_tb</code>	1

--- Matrix Multiplication Check (1x1 to 8x8) ---

```

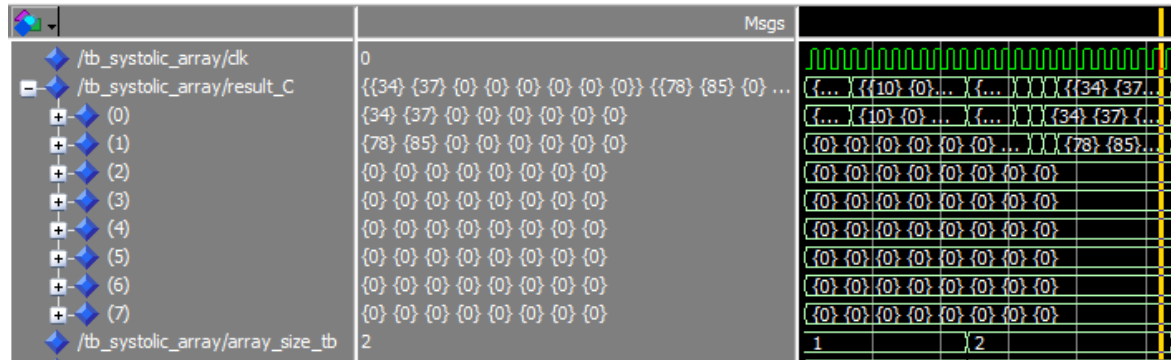
Matrix size: 1x1
Matrix A:
  1

Matrix B:
 10

Result C = A * B:
 10

Flattened C (row-major):
 10
  
```

2x2

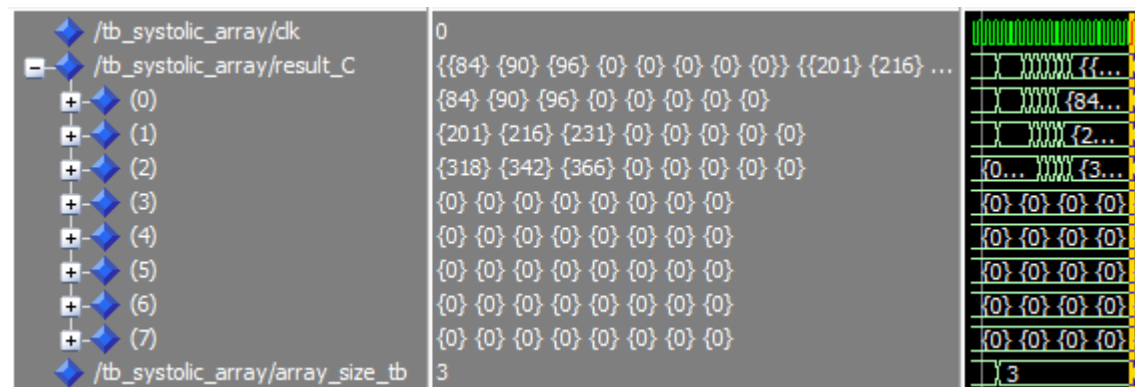


```
Matrix size: 2x2
Matrix A:
  1    2
  3    4

Matrix B:
 10   11
 12   13

Result C = A * B:
 34   37
 78   85
```

3x3

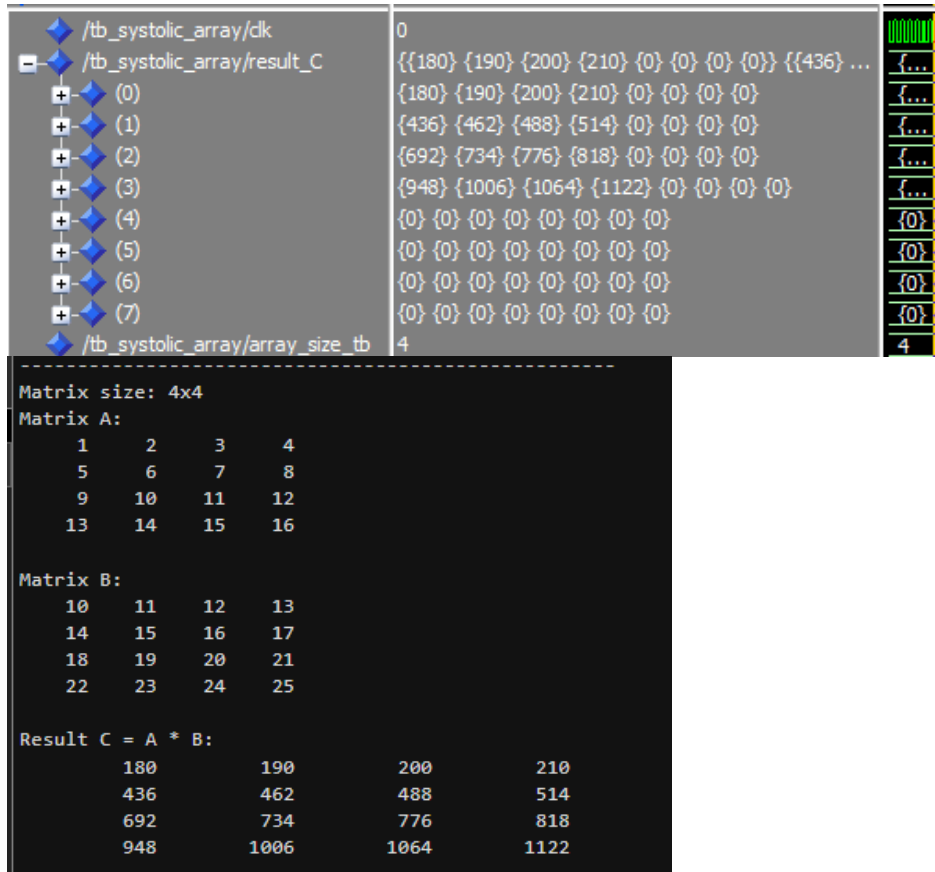


```
Matrix size: 3x3
Matrix A:
  1    2    3
  4    5    6
  7    8    9

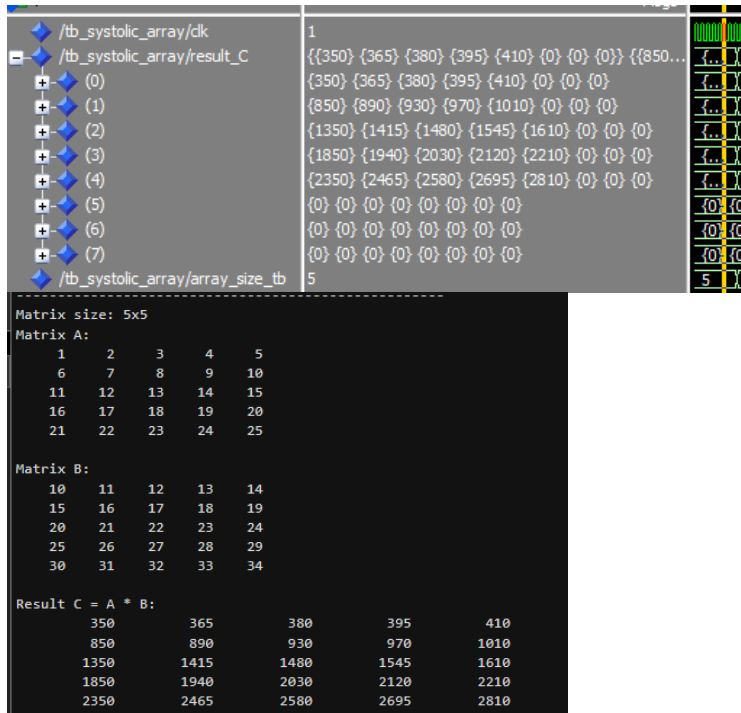
Matrix B:
 10   11   12
 13   14   15
 16   17   18

Result C = A * B:
 84   90   96
 201  216  231
 318  342  366
```

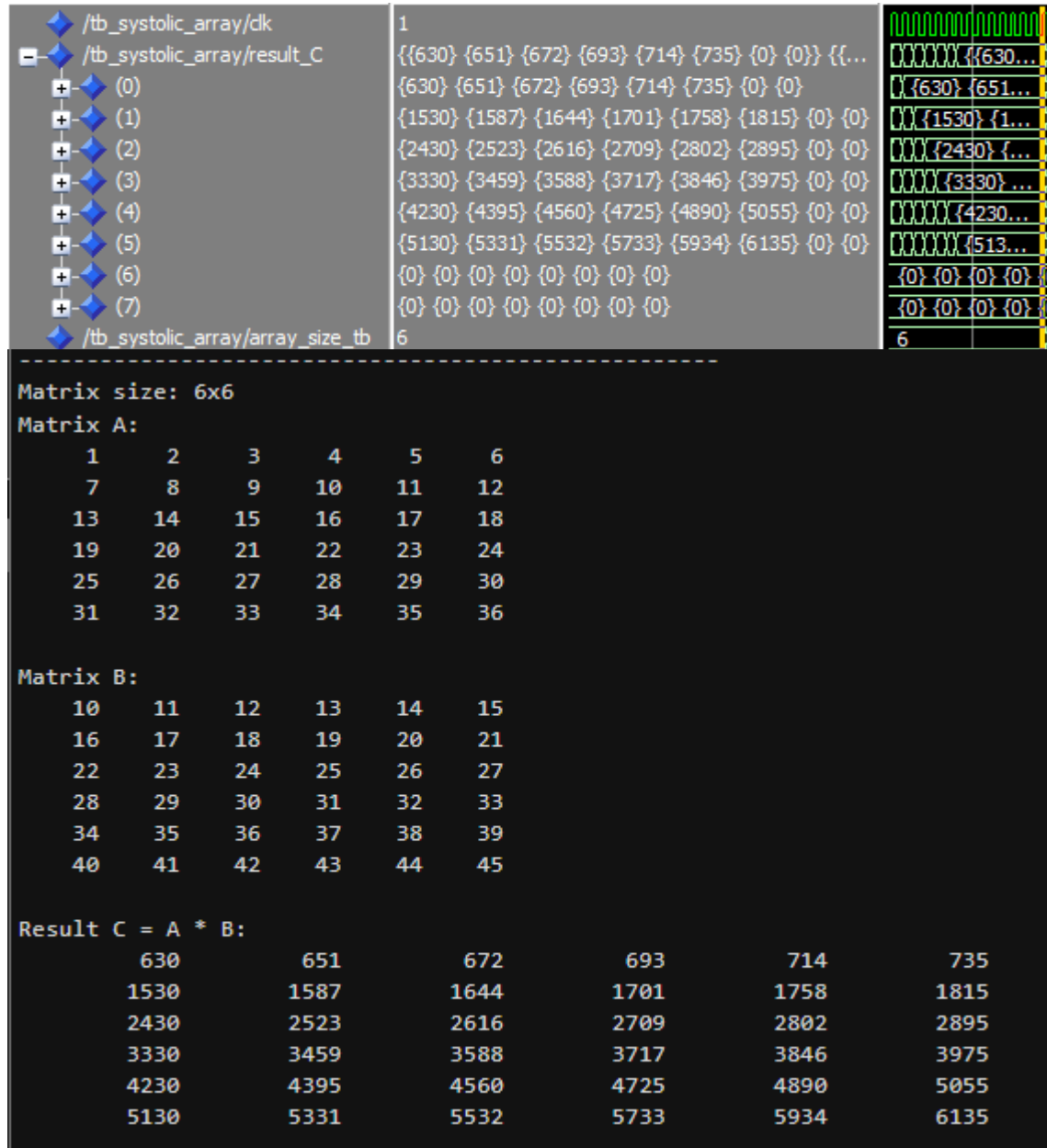
4x4



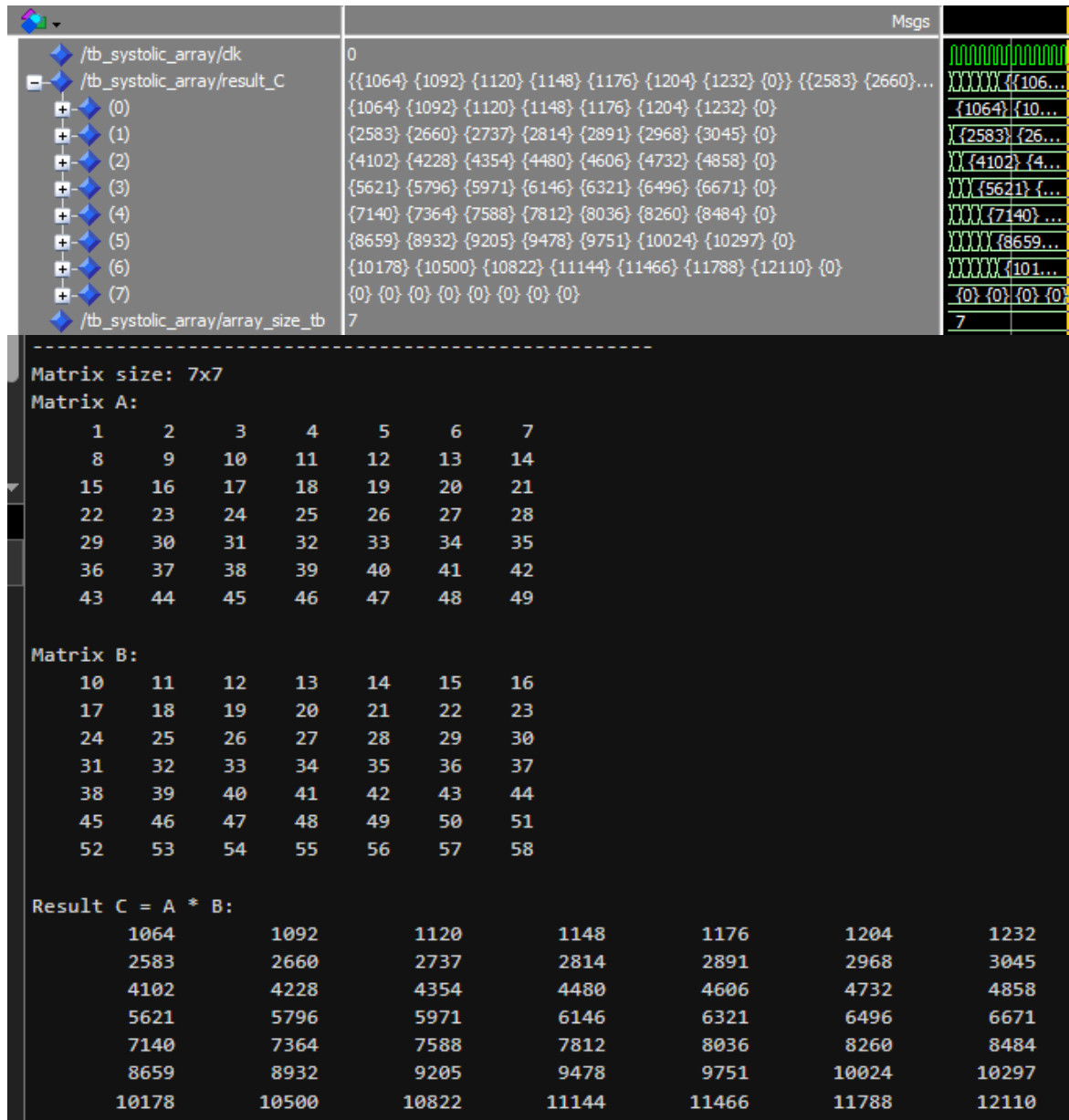
5x5



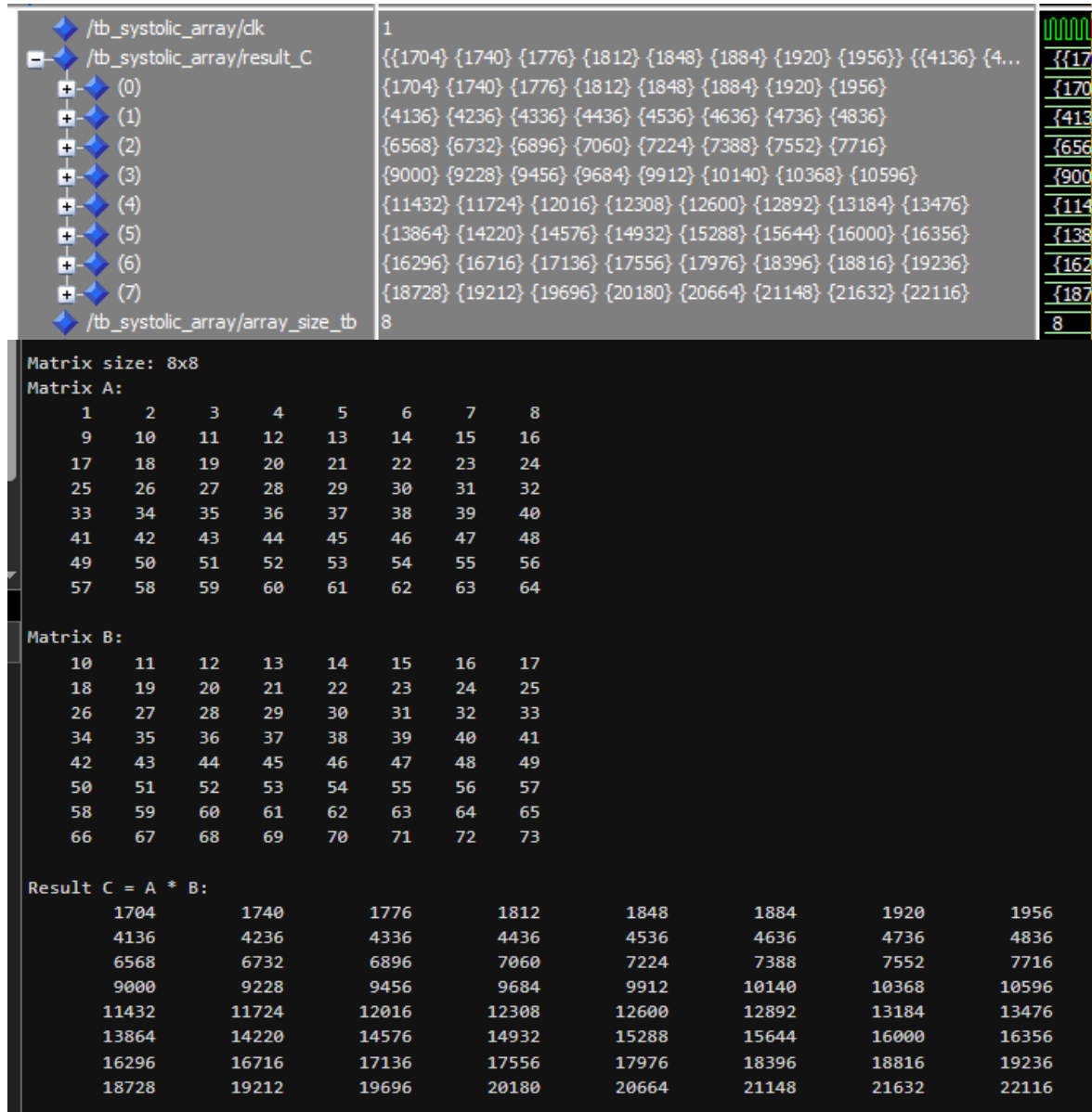
6x6



7x7



8x8`



Control Unit:

- Needs to add a mask to the data/weight inputs to make it fit into the 8x8

Conclusion:

- Now we have allowed space for an 8x8 systolic array.
- If we give an NxN input where $N = [1..8]$, the systolic array automatically adapt to it
- It is now confirmed that executing the MAC operation and passing the data/weights occur in one clock cycle
- The formula found – $\text{clock_cycles} = 3n-2$ – has been proven in the simulation. This tells us that the systolic array will NOT execute any operation (i.e PE is off) when it is not needed. This has minimised our energy consumption to the very least.

Thoughts for the future:

- How can we utilise hardware better? If we have a 8x8 systolic array but only using 4x4, can we make the systolic array run four 4x4 in parallel, speeding up the computations? Or can we re-route or change how the systolic arrays are arranged?
- Chunking / blocking? Look into the .rbf / DPR paper (although might be a bit difficult)
- Is there a better way to do MAC operations?

Optimising Systolic Array

Objective: Re-read research papers to find how we can utilise the hardware better

Systolic Array Design for Efficient FPGA Implementation of CNN Accelerators: Power and Area Optimizations

- “heterogenous systolic arrays are especially advantageous for depth wise convolutions, which involve lesser computations”
- Dynamic precision handling – bit-width of operations is adjusted based on the data requirements
- Reducing precision for simpler data/less complex cnn layers, then less power consumption and computational complexity
- Systolic is used for multiple types of operations so better resource utilisation (better for edge devices)
- To reduce external memory, buffering and data reuse techniques are employed (because accessing external memory is a major contributor to power consumption)
- Power gating (shuts off idle units) and clock gating

Planaria: Dynamic Architecture Fission for Spatial Multi-Tenant Acceleration of Deep Neural Networks

- Focus on multi-tenancy in the architecture design of DNN accelerators
- “For example, speech recognition and voice synthesis systems tend to require multiple models in deployment and can significantly benefit from multi-tenancy and co-location” But our use case is for objection recognition so I’m wondering if this is even relevant

Developing and Implementing Dynamic Partial Reconfiguration for Pre-Emptible Context Switching and Continuous End-To-End Dataflow Applications

- Reconfigurable at run time without having to reprogram
- Multitasking by context switching is possible on an FPGA by saving and restoring the context of the registers, flip-flops, and memory
- PDR uses a wrapper around the reconfigurable region that holds the input signals during the reconfigurable process to prevent operation in that region and limit Single Event Upsets (SEU)
- Prevents the compiler from using the space in the reconfigurable design region for other parts of the design
- Our current implementation isn’t even assigning parts of the fabric nor are we region locking (<https://www.intel.com/content/www/us/en/docs/programmable/683641/21-3/regions.html>). We are only controlling the dataflow but not the array itself
- To actually change the 8x8 static array to a smaller array, then we would need to do DPR, where its each synthesised and placed in a reconfigurable region
- Our design, by masking, is still saving power, but there is still leakage power consumed (which makes DPR more efficient)

Secure Energy-Efficient Implementation of CNN on FPGAs for Accuracy Dependent Real Time Task Processing

- Embedded Memory Blocks (EMBs)
- To utilise EMBs, we have to determine the available memory spaces and divide it into several memory blocks that can be loaded with pre-computed data.
- Pre-computing means instead of using systolic arrays to do MAC operations, we do this separately and save all the possibilities into a memory block

- This can be useful to use as the weights/data are fixed, especially for the early layers, and we can mix this with some systolic arrays if needed

Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks
<https://ceca.pku.edu.cn/media/lw/b73586e2ac7b5f8d63e8e584f398f17f.pdf>

•

Hardware Utilisation

Technique	How	Relevance
Planaria, Data-Mirroring	Running multiple models in one architecture (“multi-tenancy”)	Not relevant Edge devices are likely to only run one model at a time Keeps it lightweight and consumes less power
Dynamic Partial Reconfiguration (DPR) – Beasley	Allows for reconfiguration of FPGA architecture at run time without having to reprogram.	Relevant but only for ideas not execution Executed in a FPGA we are familiar with
Embedded Memory Blocks (EMBs) <i>Secure Energy-Efficient Implementation of CNN on FPGAs for Accuracy Dependent Real Time Task Processing</i>	Storing weights and immediate results in on-chip EMBs to reduce accessing external memory	Relevant Reduces power consumption
Batching Multiple Inputs	Processing multiple input samples simultaneously fills systolic array pipelines more full, increasing parallelism	

Objective 2: Research about ML in depth to find why we have sparsity in the matrices and how the systolic array receives the matrices

Vanishing Gradient Problem

<https://www.engati.com/glossary/vanishing-gradient-problem#:~:text=exploding%20gradient%20problem?-,What%20is%20vanishing%20gradient%20problem?,layer%20to%20the%20input%20layer.>

- Occurs during training DNNs
- Gradients are used to update the network and this becomes extremely small or “vanish” when they are backpropagated from output to earlier layers
- Backpropagation algorithm calculates gradient by propagating the error from the output layer to the input
- Consequences of DGP: slow convergence, network getting stuck in low minima, and impaired learning
- Issue arises in deep structures with lots of layers: activation functions like ReLU can help with this

- Can be detected when: higher layers change but lower layers barely change, model weights become 0 (this is what causes the sparsity in our context)
- What are activation functions?
 - Rectified Linear Unit (ReLU) allow for better gradient flow

Summary:

The gradient is calculated to tell each layer how much it should change to reduce the final loss. To calculate this, $\text{gradient} = \text{slope} * \text{output error of the layer ahead of it}$ (keep in mind this is back propagation). Each layer has its own slope and the gradient is calculated for the weight and neuron. When the gradient is applied to all layers, the error gets smaller and smaller as it gets closer to the input layer causing the vanishing gradient layer. The purpose of the ReLU is deciding whether or not the gradient should be applied. If the neurons are activated (i.e found a feature), then the gradient is applied to enhance that feature. Otherwise, the ReLU does not apply the gradient, keeping the weights unchanged, and blocks the rest of the back propagation layers from changing its weights as well.

- **ReLU's derivative is:**

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

How does the system work?

- The pre-trained model will provide the initial image matrix (data matrix) and all the weight matrices of each convolutional layer. Only in the first layer is the image provided as the data matrix. The weight matrices are stored in memory.
- These are passed onto the control unit where it will decide what the systolic array needs to do to process the data.
- The systolic array will perform the MAC operations. Once completed, this will be stored in memory as well. The output of systolic array becomes the data input of the following layer. For example, the inputs of layer 2 is: systolic array output of layer 1 and the weights of layer 2. This becomes a loop until the systolic array has completed all the layers.
- Once all the layers are completed, we need to do some processing with a CPU (maybe NIOS?) that will make the final prediction.

Question:

- If the inputs are not NxN size nor are the data and weight matrices the same size, what happens?

Monday 21st July – Wednesday 23rd July 2025**Time: All Day**

Back at it again!

Objective: Complete writing technical report by Wednesday Night

Research Paper:

Sense: Model-Hardware Codesign for Accelerating Sparse CNNs on Systolic Arrays

- Found issue is the irregularity of sparsity reduces performance
- Reduces DRAM access by 1.17-1.8x compared to Swallow
- “Since data movement consumes much more energy than computation”
- Focused on data reuse
- “For IFMs, approximately half of their elements are zeros because of the widely used nonlinear function rectified linear unit (ReLU); for weights, the fault tolerance of CNN allows of a large number of redundant zeros, which inspires researchers to develop pruning and quantization methods”
- “But sparse IFMs and weights can be irregular and fragmented, which leads to lower memory access efficiency.”
- Why systolic array: “Second, systolic arrays propagate data horizontally and vertically with simplified inter-PE connection, which can highly reuse data and reduce data movement consumption. Finally, communication with the outside occurs only at the boundary PEs, providing huge bandwidth saving.”
- Why systolic array good: “Due to the structured dataflow, the control unit only takes up a small portion of system hardware cost, achieving high resource utilization.”
- Issue with sparsity in systolic array: “However, the irregularity of Fig. 2. Systolic array architecture. Fig. 3. (a) Imbalanced and (b) balanced loads of sparse weights in the systolic array. sparsity conflicts with the structured dataflow in systolic array, which may induce imbalanced workloads, causing low PE utilization and insufficient acceleration.”
- Load-balancing weight pruning flow: basically analyse each kernel, make the two kernels have the same number of non-zero elements by pruning the least important element to zero, then compare accuracy, if accuracy not lost then continue to prune the same way
 - how does it determine what is the least important element? CNNs contain many redundant weights, and with smaller absolute value of weight comes less importance.
 - Having an equal sized kernel leads to better hardware utilisation and 1.5x speed
- Channel clustering:
 - Finds a common characteristic in the channel (i.e. sparsity) and groups them together
 - The cluster will do computations together
- This system loads in the pruned weights
- On-Chip Buffer contains the input, weight, and output buffers, two BRAMs
- What is Ping-Point buffers?

PRUNING CONVOLUTIONAL NEURAL NETWORKS FOR RESOURCE EFFICIENT INFERENCE

- Prunes the entire feature maps so resulting network can run efficiency even on embedded devices
- Uses second-order taylor expansion to select parameters for deletion
- Criteria for pruning
 - Minimum weight: have a minimum weight threshold
 - Activation: ReLU functions tend to zero out values (like negative values). If the kernel is consistently zeros, then we can assume that it is not relevant to the prediction without risking accuracy
“it is reasonable to assume that if an activation value (an output feature map) is small then this feature detector is not important for prediction task at hand”
 - Taylor expansion: ????

How does changing the kernel values not affect inference?

CNNs contain many weights and layers with lots of computations. pruning one weight matrix would not have as a large impact as we think we have. for example, if one node is inaccurate, in the scale of CNNs, it actually change the prediction. additionally, whilst the weight values do matter, the purpose of the CNN is to have an accurate prediction. lets say we have a control model and the pruned model, if they have different weight values at inference, but can recognise the same object, then both are still accurate

Compiling on Quartus:

- Fmax is at

Slow 1100mV 85C Model Fmax Summary				
<<Filter>>				
	Fmax	Restricted Fmax	Clock Name	Note
1	256.08 MHz	256.08 MHz	clk	

- However we are failing the timing requirements because we have not set it up properly so Quartus is assuming that we are running it at the highest speed


```

169085 No exact pin location assignment(s) for 3106 pins of 3106 total pins. For the list of pins please
169281 There are 1026 IO input pads in the design, but only 458 IO input pad locations available on the d
169282 There are 2080 IO output pads in the design, but only 458 IO output pad locations available on the
11798 Fitter preparation operations ending: elapsed time is 00:00:01
      
```
- We create a constraints.sdc file that sets the 50Mhz:


```

# Define a 50 MHz clock on the top-level input port 'clk'
create_clock -name clk -period 20.000 [get_ports clk]
      
```
- Now the output of the timing analyser is

Compilation Report - testing_compilation

Table of Contents

- Parallel Compilation
- SDC File List
- Clocks
- Slow 1100mV 85C Model
 - Fmax Summary**
 - Timing Closure Recommendation
 - Setup Summary
 - Hold Summary
 - Recovery Summary
 - Removal Summary
 - Minimum Pulse Width Summary
 - Metastability Summary
- Slow 1100mV OC Model
- Fast 1100mV 85C Model
- Fast 1100mV OC Model
- Multicorner Timing Analysis Summary
- Advanced I/O Timing
- Clock Transfers
- Report TCCS
- Report RSKM
- Unconstrained Paths

Slow 1100mV 85C Model Fmax Summary

<<Filter>>

	Fmax	Restricted Fmax	Clock Name	Note
1	118.64 MHz	118.64 MHz	clk	

This panel reports FMAX for every clock in the design, regardless of specified clock periods. FMAX is only computed for paths where destination registers or ports are driven by the same clock. Paths of different

- This means that our simulation from ModelSim is valid because it is running at 50MHz

Questions:

- How does the systolic array's output become its own input through the layers?
- Basically how does the kernel / filtering thing work (if the image is 128x128 and our kernel is 3x3, what happens?)

Gap:

- Why systolic array: "Second, systolic arrays propagate data horizontally and vertically with simplified inter-PE connection, which can highly reuse data and reduce data movement consumption. Finally, communication with the outside occurs only at the boundary PEs, providing huge bandwidth saving." – SENSE
- Why systolic array good: "Due to the structured dataflow, the control unit only takes up a small portion of system hardware cost, achieving high resource utilization." – SENSE
- "Sparsity conflicts with the structured dataflow in systolic array, which may induce imbalanced workloads, causing low PE utilization and insufficient acceleration" – SENSE
- "Computation of convolutional neural network (CNN) requires a significant amount of memory access, which leads to lots of energy consumption. As the increase of neural network scale, this phenomenon is further obvious, the energy consumption of memory access and data migration between on-chip buffer and off-chip DRAM is even much more than the computation energy on processing element array (PE array)." From Reconfigurable Architecture and Dataflow for Memory Traffic Minimization of CNNs Computation

What can we do about this gap?

- Load-Balancing Pruning – SENSE

- Memory??
- FPGA Fabric??

Week 3

04/08/25 – 10/08/25:

Monday 4th August 2025

Time: All DAY

Major Catch-Ups on doubts

Objective: Fill in the conceptual gaps

How are we going to utilise ML in our project?

- Once we have a pre-trained model, we can actually extract CNN layers on Python.
- <https://www.datacamp.com/tutorial/cnn-tensorflow-python>
- Use the BRAM of the FPGA to locally store the numerical weights and activation layers

Plan for the next 2 weeks:

1. Get a version of AlexNet on Python
2. Extract the numerical weights and activation layers as matrices. Most likely just extracting 1 layer as a test-case.
3. Figure out how to store the matrix into the FPGA (this is conceptually not implementing)
4. Make a software-only case where Python will execute the layer and time it
5. Figure out how we do the same with the current implementation of the systolic array. See what changes need to be made

So, what does this mean in the context of our project?

We have AlexNet running on Python that allows us to extract one fully connected layer of weight matrix. As this has been tiled, we can make it fit into our 8x8 systolic array. Our current activation layer is only utilising 1xN. This is actually not ideal, we would be better off running 8 batches in parallel so we can continue with our matrix multiplication of our systolic array.

To do:

- Weight stationary
- Figure out what python code means

Week 5

18/08/25 – 24/08/25:

Monday 18th August 2025

Time: All Day

Getting back to it

Objective: Convert .mif to 2D array

```
% Data Matrix (A)
A = [ 1  2  3  4  5  6  7  8;
      9 10 11 12 13 14 15 16;
      17 18 19 20 21 22 23 24;
      25 26 27 28 29 30 31 32;
      33 34 35 36 37 38 39 40;
      41 42 43 44 45 46 47 48;
      49 50 51 52 53 54 55 56;
      57 58 59 60 61 62 63 64];

% Weight Matrix (B)
B = [ 8  7  6  5  4  3  2  1;
      16 15 14 13 12 11 10  9;
      24 23 22 21 20 19 18 17;
      32 31 30 29 28 27 26 25;
      40 39 38 37 36 35 34 33;
      48 47 46 45 44 43 42 41;
      56 55 54 53 52 51 50 49;
      64 63 62 61 60 59 58 57];
```

```
>> sandpit
Accumulated value of each PE (row i, column j):
      1632      1596      1560      1524      1488      1452      1416
      3936      3836      3736      3636      3536      3436      3336
      6240      6076      5912      5748      5584      5420      5256
      8544      8316      8088      7860      7632      7404      7176
     10848     10556     10264     9972     9680     9388     9096
     13152     12796     12440     12084     11728     11372     11016
     15456     15036     14616     14196     13776     13356     12936
     17760     17276     16792     16308     15824     15340     14856
```

Friday 12th September 2025**Time: All Day**

Sparsity Algorithm

Objective: Find a way to combat sparsity that is not during pre-processing (NIOs) and does not interfere with the systolic array pipelining

FPGA Prototyping of Systolic Array-based Accelerator for Low-Precision Inference of Deep Neural Networks

- “If structured sparsity exists in the granularity of the 128×128 square block, our inference accelerator can skip the entire computation for the corresponding block, thereby significantly minimizing the computation cycle”

Sparse-TPU: Adapting Systolic Arrays for Sparse Matrices

- The way packing works here is that you look at a column. You check the column on the right and push it onto the first column. If there are any collisions, then a new row is created.
- Keep track of this because we need to rearrange the input the same way so that we can mathematically do the MAC operations correctly. This basically just skips out on the zeros in the systolic array.

Thursday 18th September 2025**Time: All Day****Objective:** Find Sparsity Handling

Gating and Packing idea:

- We don't want to start off with a packing scheme then change up the systolic array pipeline
- The current systolic array handles a uniform $N \times N$ set
- Instead of changing the architecture of the systolic array, we would rather handle this on the control unit's end
- We want to remove the zero elements / sparsity in the matrix from the ML
- With gating, the systolic array still oversees the zero elements but it still 'processes' it. Instead we want to get rid of the zeros before it comes in
- However, when the zeros are removed, we would be sending LOCATIONs of the useful instead

Gating with Mask:

- When looking at the input matrices, we mask the non-zero elements with ones and find the positions of these
- We send this to the systolic array so it knows beforehand what PEs to turn on or off
- In comparison to the baseline systolic array, the original still does a process where it checks if it should be on, but with this, the control unit knows already

Monday 22nd September 2025**Time: All Day**

Read SCNN Paper and attempt an implementation through code

SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks

- the SCNN dataflow only delivers weights and activations to the multiplier array that can all be multiplied by one another in the manner of a Cartesian product. Furthermore, the activation vectors are reused in an input stationary [6] fashion against a number of weight vectors to reduce data accesses. Finally, only non-zero weights and activations are fetched from the input storage arrays and delivered to the multiplier array.
- SCNN doesn't do the typical MAC operation where the multiplied output is directly multiplied in the PE. Instead, it tracks the output coordinate
- Their methods of exploiting sparsity:
 - Compressing data: Encoding the sparse weights and/or activations provides an architecture an opportunity to reduce the amount of data that must be moved throughout the memory hierarchy. It also reduces the data footprint, which allows larger matrices to be held a given size storage structure.
 - Eliminating computation: For multiplications that have a zero weight and/or activation operand, the operation can be data gated or the operands might never be sent to the multiplier. This can save energy consumption or both time and energy consumption, respectively.

Code Changes:

- Building upon the power-gating method where we are sending an enable mask to the systolic array if one of the input is going to be zero
- Extracting an 8x8 tile from one of the ML layer and saving it as .mif
- Using Python to extract a 8x8 matrix from AlexNet model
- Using Matlab to read the .mif and convert to 2D array, ready for pre-processing
<https://pypi.org/project/mif/>

Basic Column Packing Algorithm (from Sparse-TPU paper)

```
% This is a script to call our MIF reader function and pack the matrix
clear;
clc;

% 1. Define the path to your .mif file
mif_filepath = 'C:/Users/iamkr/Documents/part-4-project/Systolic Array
(packing)/mif/basic_packing_algo.mif';
N = 8; % Define the matrix size

% 2. Call the function to get the 1D array
my_data_array_1d = mif_to_1d_array_matlab(mif_filepath);

% 3. Reshape the 1D array into a 2D matrix
my_data_array_2d = reshape(my_data_array_1d, [N, N]);
```

```

disp('Original 8x8 Array:');
disp(my_data_array_2d);

% --- Corrected Column Packing Algorithm ---
packed_groups = {};

for i = 1:size(my_data_array_2d, 2)
    candidate_col = my_data_array_2d(:, i);

    if ~any(candidate_col)
        continue;
    end

    was_packed = false;

    for g = 1:length(packed_groups)
        % Corrected collision check: Element-wise multiply.
        % If the sum is 0, no non-zero elements overlapped.
        if sum(packed_groups{g} .* candidate_col) == 0
            % No collision, so merge
            packed_groups{g} = packed_groups{g} + candidate_col;
            was_packed = true;
            break;
        end
    end

    if ~was_packed
        packed_groups{end+1} = candidate_col;
    end
end

% --- Reassemble for display ---
if ~isempty(packed_groups)
    num_packed_cols = length(packed_groups);
    packed_array = zeros(size(my_data_array_2d, 1), num_packed_cols);
    for i = 1:num_packed_cols
        packed_array(:, i) = packed_groups{i};
    end
else
    packed_array = [];
end

disp('Packed Array:');
disp(packed_array);

```

Command Window

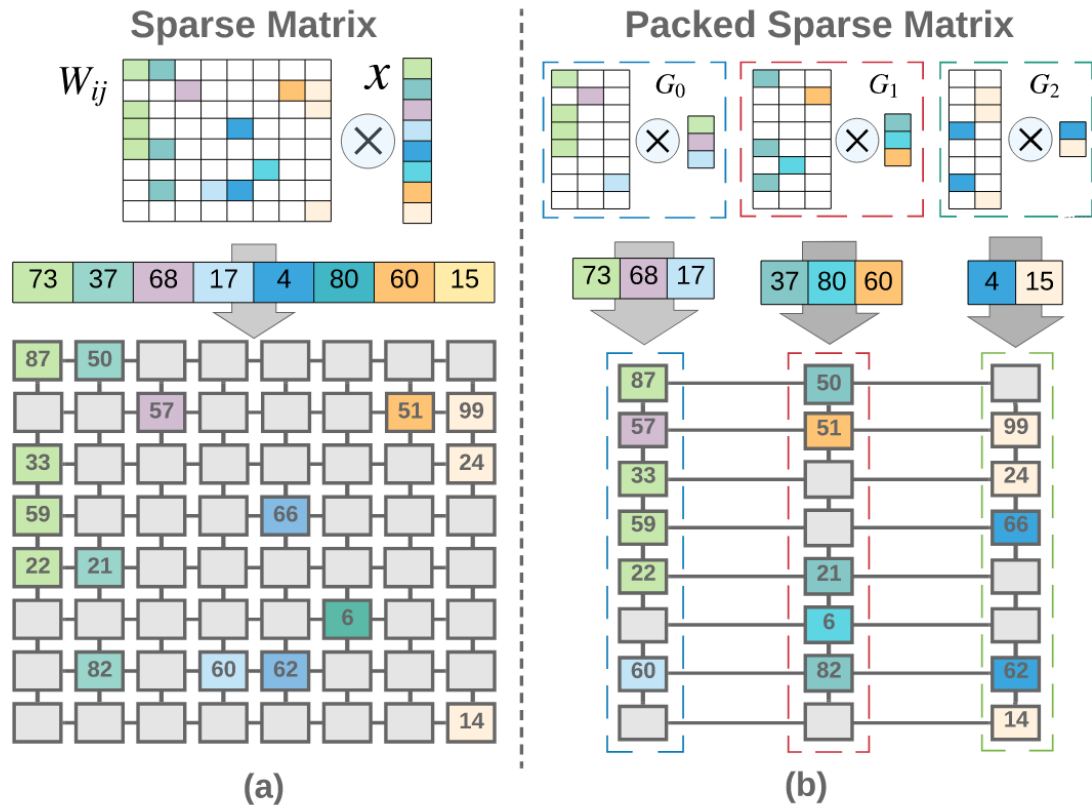
```
>> mif_reader
```

```
Original 8x8 Array:
```

87	50	0	0	0	0	0	0
0	0	57	0	0	0	51	99
33	0	0	0	0	0	0	24
59	0	0	0	66	0	0	0
22	21	0	0	0	0	0	0
0	0	0	0	0	0	6	0
0	82	0	0	60	62	0	0
0	0	0	0	0	0	0	14

```
Packed Array:
```

87	50	0
57	51	99
33	0	24
59	0	66
22	21	0
0	6	0
62	82	60
0	0	14



This version of the code is slightly wrong, specifically with the second to last row. Will try a new method

Simple Packing Idea:

- Count the amount of non-zero values in a matrix
- Find the best matrix size to remap it to. For example for 15 non-zero elements, it can fit in a 4x4 matrix.
- Keep track of the original position to the new one using cell arrays
- But by providing the .mif, we actually know where the original position of the numbers are

Wednesday 24th September 2025

Time: All Day

Working Row/Column Stripping method

- This only works for structured sparsity where the entire row/column must be all zeros
- The way it works is that in the Python file, we look at the matrix and get rid of all the rows/columns with zeros
- We print out the VHDL code for the test bench here as well
- In the test bench, model sim will display without the sparse row/columns but we need to pass this back to software to “re-arrange” it back
- The stripping method works because we are basically shifting the row and column, so we are just skipping the entire row and column, which keeps the maths consistent with the systolic array
-

Thursday 25th September 2025

Time: All Day

NPU Wrapper

- When synthesising onto the board, the code has issues where we can't actually compile it because of pin overflow.
- DE1-SOC has 256KB memory.
 - Input: 64 element matrix * 2 matrix * 1 byte (int8) = 128 Bytes
 - Output: 64 element matrix * 4 bytes (int32) = 256 Bytes
 - Totalling = 385 Bytes

Friday 26th September 2025

Time: 1PM

Tiling:

- Store the whole layer in the SDRAM as tiles
- In Python, we tile (for example 64x64) into 8x8 and save as MIF. Then we send through UART
- By tiling, we are able to split the sections of the layers, and see that there are sections of rows/columns of zeros, which we may not see if all the way zoomed out

To do:

- Function that can send MIF through UART using PySerial
- Check why Activation layer is 8x8 whilst Weight is 64x64
- Keep track of the stripped column/rows and put it back
- Tile and store the meta data that includes the coordinate of the zero elements OF THE TILED AND STRIPED
- Fix the stripping algo by letting it take sizes from matrices (Pratham)
- Is it possible to calculate the active row/cols in NIOS instead? (Pratham)

Complete To-Do list above

AlexNet:

- Eight Layers: 5 convolutional, some max-pooling layers, three fully connected layers
- The avgpool flattens the activation layer

Check why Activation layer is 8x8 whilst Weight is 64x64

- Due to incorrect code. This is now fixed

Issue with the current Stripping Algorithm:

- The issue comes from the fact that we are doing row stripping on both. For whatever reason, when we row strip BOTH the activation and data matrices, there is mismatch happening between the multiplication.
- Looking at the first PE, row of A, and column of B alone:
 - We are expecting the behaviour of $90 \times 13 + 90 \times 4 + 90 \times 27 = 3960$. However, because we have stripped a row in data, we get $90 \times 13 + 90 \times 4 + 92 \times 27 = 4014$
 - This indicates what when we row strip from the data matrix, the activation matrix's column is somehow one column behind

Modified Stripping Algorithm:

- Algorithm now considers both matrices at the same time when getting rid of a row/column. What this does is if it gets rid of a row in Matrix A, it checks if we get rid of the column in Matrix B
- This maintains the positions correctly so we can correctly apply the MAC operation in the systolic array.
- The downside to this (compared to the original algorithm) is that there is less row/column stripping happening. This means that we are not reducing the clock cycles as much as we hope, but at least we know it could work for real matrices.
- An important note is that since the activation matrix is always going to be 1x8, the weight matrix will always be 8x8 UNLESS we get rid of a column.
 - This is because we can't get rid of rows in the weightmatrix since there is a number in all the columns of the activation matrix
 - So, in terms of reduction, we have to hope that there is an empty column in the weight

Week 9

29/09/25 – 05/10/25:

Monday 29th September 2025

Time: 4PM

Combine all the Python Code into one file

- Made the separate files into helper functions
- Simulated a systolic array behaviour

Week 10

06/10/25 – 12/10/25:

Monday 6th September 2025

Time: 2PM

Memory!

Objective: Attempt to connect the MIF to simulation

My approach to this is the same as the To-Do in the meeting minutes

- Instantiate the buffers in the TB
- Read the buffers separately. Store these values into an array.
 - Create an FSM with START, READ, END states.
 - Test with a simple 8x8 array
 - Test with a sparse matrix
- Hardcode the active_rows and active_cols values into the TB itself. Can later add this to the MIF if required / have the time.

Current Implementation of tb_memory.vhd

LatencyTest Process: FSM Logic

IDLE: waits for rising edge

APPLY_RESET: asserts raising edge for 2 clock cycles

LOAD_DATA:

- Read data and weight simultaneously
- Address begins at zero
- Index is shifted by 2
 - Noticed during test that we would add numbers to the matrix two positions early
 - Hardcoded solution for now
 - Need to figure out root cause (potentially latching issue)
- Target_row: index/N, target_col: index mod N
- If the counter reaches the end of the MIF, move to next state

WAIT_FOR_COMPLETION:

- waits for the process to end
- feels like a redundant state

Additional Updates to the code:

- control unit now has a ready signal, asserted when we are done reading the MIF files. This is because it will start counting as soon as rising edge starts, meaning that we are not accessing the matrix in time

- $3N-2$ is the theoretical clock cycle formula, however, since most of the testing we have done are with 1D arrays (post sparsity processing), this formula has become $2n$. NOTE: this is only for 1D arrays

Next Steps:

- Packing method: allows us to read multiple lines in the MIF, reduces memory reads
- Generate varying MIF files from AlexNet
 - Start with no sparsity, building up to more sparsity
- Track throughput and systolic array size to determine performance
 - Can calculate energy efficiency based on this

Wednesday 8th September 2025

Time: ALL DAY

Testing

Objective: Get Data !

Notes:

- we found that the stripping algorithm does not remove any sparse row or column
- when the algorithm is deciding whether or not to strip a row or column, it must check the if the inner dimensions are
- for example, if the data matrix can be compacted down to 5×3 but the weight matrix is limited to 8×8 , the algorithm does not strip the columns in the data matrix. This means that the size will still be 5×8 for oblige with matrix multiplication rules
 - This means that in modelsim, we will still process those zero columns, but since we are accumulating zeros to the PE, the results matrix will stop updating much earlier on
 - Although it might not update its value on modelsim, it is important to note that these processes are still happening
 - This may not be the most efficient algorithm but the latency is still less than $3N-2+1$.
 - The table should show this
- For a completely sparse data matrix, the latency is still 15 clock cycles. This feels wasteful but it is better than 23.
- Can't column strip. This means that if the structured sparsity is at the columns, and the weight is dense (i.e. all 64 values), then we must utilise all 23 clock cycles.

Thursday 9th September 2025

Time: ALL DAY

NPU Wrapper

Objective: Create an NPU wrapper to be synthesised to Quartus

Observations:

- MAC Count: counting the active PEs in the output (i.e. output dimension – PE with zero output)
 - Noticed that none of the tested output contained zero in the output dimension

- NPU Wrapper:
 - Instantiated top_level_systolic_array and the data buffers
 - Because we have limited pins in DE1-SoC, we cannot map the systolic_array 8x8 output to the GPIO pins
 - We modified this by sending the 32-bit value through an output bus
- Compilation Report:
 - No DSPs are used as design is simple and small
 - Processing Element no longer does zero-gating.
 - This sped up compilation from 4:12 to 3:30. We can get away with this because accumulating zero does not change the result AND using comparators for the if condition is costly for hardware.
 - Furthermore, the active matrix is sparse, but the weight matrix is dense. This means that the output matrix (as mentioned above) will never be zero. Although accumulating zero is redundant, going through the if condition and doing the MAC operation is done in the same clock cycle
 - Since this does not change the timing analysis or hardware utilisation (in terms of the slack value), then this decision makes more sense
 - Uses 1 RAM block: both data and weight buffers use 128 words x 8 bits. The size of 1 M10K block is 1024 words x 8 bits
 - No matter file we test, Quartus will synthesise it worst case. Since we are always instantiating the systolic array to be 8x8, even though we are PE-gating, it will synthesise it as 8x8 regardless

Pratham's Testing:

--- Sparsity Analysis Summary ---

Total images processed: 156

Average Original Latency (dense 8x8): 22.00 cycles

Average Compacted Latency (after stripping): 9.07 cycles

Latency Reduction: 58.75%

--- ALEXNET MODEL PROFILING ---

Total MACs for AlexNet: 0.71 GMACS

to conclude:

a latency reduction of 58.75% would mean that the NPU would only use 41.25% of the original cycles

Acceleration factor: $1/0.4125 = 2.42$

total cycles saved $710,000,000 * 0.5875 = 417$ Million Cycles

THIS MEANS WE HAVE (FOR THIS DATASET) A SPEEDUP OF 2.42

Saturday 11th to Sunday 12th October

Time: All Day

Connecting UART

Objective: Figure out a way to transmit data through UART

Purpose:

- Find a way to output something from the FPGA to a terminal

HPS / USB-UART:

- Has a mini USB-B port to USB-A port
- Requires the ARM processor
- Hard to configure and time consuming
- Configured QSF file to connect to all the peripherals and GPIO

Pico:

- Found a Youtube Video: <https://www.youtube.com/watch?v=fMmcSpgOtJ4>
- Use that and this GitHub as a reference: <https://github.com/NoamElbaum/VHDL-UART>
- The idea is that, I use two GPIO pins from the FPGA as the RX and TX ports of the UART.
- I am using the UART code from the video/GitHub and modifying it to fit our code
- However, an issue found is that we a TTL to USB adapter so we can actually read these pins. It was Sunday, the shop was closed, didn't want to wait 24 hours.
- BUT I found that we can actually use microcontrollers to be this middle-man instead.
- Originally, I was using the Arduino UNO R3 but these GPIO pins need 5V to be driven high, whereas the FPGA output GPIO pins only output 3.3V
- From Capstone, A Raspberry Pi Pico uses 3.3V so I went out to buy one (I have one at Uni but I'm at home)

40 Pin Header assignments for DE1-SoC

GPIO 0										GPIO 1									
GPIO_000	1	2	GPIO_001	GPIO_100	1	2	GPIO_101	GPIO_002	3	4	GPIO_102	GPIO_003	5	6	GPIO_103	GPIO_004	7	8	GPIO_104
GPIO_005	9	10	GPIO_006	GPIO_105	11	12	GPIO_106	GPIO_007	13	14	GPIO_107	GPIO_008	15	16	GPIO_108	GPIO_009	17	18	GPIO_109
GPIO_010	19	20	GPIO_011	GPIO_110	21	22	GPIO_111	GPIO_012	23	24	GPIO_112	GPIO_013	25	26	GPIO_113	GPIO_014	27	28	GPIO_114
GPIO_015	29	30	GPIO_016	GPIO_115	31	32	GPIO_116	GPIO_017	33	34	GPIO_117	GPIO_018	35	36	GPIO_118	GPIO_019	37	38	GPIO_119
GPIO_020	39	40	GPIO_021	GPIO_120	41	42	GPIO_121	GPIO_022	43	44	GPIO_122	GPIO_023	45	46	GPIO_123	GPIO_024	47	48	GPIO_124
GPIO_025	49	50	GPIO_026	GPIO_125	51	52	GPIO_126	GPIO_027	53	54	GPIO_127	GPIO_028	55	56	GPIO_128	GPIO_029	57	58	GPIO_129
GPIO_030	59	60	GPIO_031	GPIO_130	61	62	GPIO_131	GPIO_032	63	64	GPIO_132	GPIO_033	65	66	GPIO_133	GPIO_034	67	68	GPIO_134
GPIO_035	69	70	GPIO_036	GPIO_135	71	72	GPIO_136	GPIO_037	73	74	GPIO_137	GPIO_038	75	76	GPIO_138	GPIO_039	77	78	GPIO_139
GPIO_040	79	80	GPIO_041	GPIO_140	81	82	GPIO_141	GPIO_042	83	84	GPIO_142	GPIO_043	85	86	GPIO_143	GPIO_044	87	88	GPIO_144
GPIO_045	89	90	GPIO_046	GPIO_145	91	92	GPIO_146	GPIO_047	93	94	GPIO_147	GPIO_048	95	96	GPIO_148	GPIO_049	97	98	GPIO_149
GPIO_050	99	100	GPIO_051	GPIO_150	101	102	GPIO_151	GPIO_052	103	104	GPIO_152	GPIO_053	105	106	GPIO_153	GPIO_054	107	108	GPIO_154
GPIO_055	109	110	GPIO_056	GPIO_155	111	112	GPIO_156	GPIO_057	113	114	GPIO_157	GPIO_058	115	116	GPIO_158	GPIO_059	117	118	GPIO_159
GPIO_060	119	120	GPIO_061	GPIO_160	121	122	GPIO_161	GPIO_062	123	124	GPIO_162	GPIO_063	125	126	GPIO_163	GPIO_064	127	128	GPIO_164
GPIO_065	129	130	GPIO_066	GPIO_165	131	132	GPIO_166	GPIO_067	133	134	GPIO_167	GPIO_068	135	136	GPIO_168	GPIO_069	137	138	GPIO_169
GPIO_070	139	140	GPIO_071	GPIO_170	141	142	GPIO_171	GPIO_072	143	144	GPIO_172	GPIO_073	145	146	GPIO_173	GPIO_074	147	148	GPIO_174
GPIO_075	149	150	GPIO_076	GPIO_175	151	152	GPIO_176	GPIO_077	153	154	GPIO_177	GPIO_078	155	156	GPIO_178	GPIO_079	157	158	GPIO_179
GPIO_080	159	160	GPIO_081	GPIO_180	161	162	GPIO_181	GPIO_082	163	164	GPIO_182	GPIO_083	165	166	GPIO_183	GPIO_084	167	168	GPIO_184
GPIO_085	169	170	GPIO_086	GPIO_185	171	172	GPIO_186	GPIO_087	173	174	GPIO_187	GPIO_088	175	176	GPIO_188	GPIO_089	177	178	GPIO_189
GPIO_090	179	180	GPIO_091	GPIO_190	181	182	GPIO_191	GPIO_092	183	184	GPIO_192	GPIO_093	185	186	GPIO_193	GPIO_094	187	188	GPIO_194
GPIO_095	189	190	GPIO_096	GPIO_195	191	192	GPIO_196	GPIO_097	193	194	GPIO_197	GPIO_098	195	196	GPIO_198	GPIO_099	197	198	GPIO_199
GPIO_100	199	200	GPIO_101	GPIO_200	201	202	GPIO_201	GPIO_102	203	204	GPIO_202	GPIO_103	205	206	GPIO_203	GPIO_104	207	208	GPIO_204
GPIO_105	209	210	GPIO_106	GPIO_205	211	212	GPIO_206	GPIO_107	213	214	GPIO_207	GPIO_108	215	216	GPIO_208	GPIO_109	217	218	GPIO_209
GPIO_110	219	220	GPIO_111	GPIO_210	221	222	GPIO_211	GPIO_112	223	224	GPIO_212	GPIO_113	225	226	GPIO_213	GPIO_114	227	228	GPIO_214
GPIO_115	229	230	GPIO_116	GPIO_215	231	232	GPIO_216	GPIO_117	233	234	GPIO_217	GPIO_118	235	236	GPIO_218	GPIO_119	237	238	GPIO_219
GPIO_120	239	240	GPIO_121	GPIO_220	241	242	GPIO_221	GPIO_122	243	244	GPIO_222	GPIO_123	245	246	GPIO_223	GPIO_124	247	248	GPIO_224
GPIO_125	249	250	GPIO_126	GPIO_225	251	252	GPIO_226	GPIO_127	253	254	GPIO_227	GPIO_128	255	256	GPIO_228	GPIO_129	257	258	GPIO_229
GPIO_130	259	260	GPIO_131	GPIO_230	261	262	GPIO_231	GPIO_132	263	264	GPIO_232	GPIO_133	265	266	GPIO_233	GPIO_134	267	268	GPIO_234
GPIO_135	269	270	GPIO_136	GPIO_235	271	272	GPIO_236	GPIO_137	273	274	GPIO_237	GPIO_138	275	276	GPIO_238	GPIO_139	277	278	GPIO_239
GPIO_140	279	280	GPIO_141	GPIO_240	281	282	GPIO_241	GPIO_142	283	284	GPIO_242	GPIO_143	285	286	GPIO_243	GPIO_144	287	288	GPIO_244
GPIO_145	289	290	GPIO_146	GPIO_245	291	292	GPIO_246	GPIO_147	293	294	GPIO_247	GPIO_148	295	296	GPIO_248	GPIO_149	297	298	GPIO_249
GPIO_150	299	300	GPIO_151	GPIO_250	301	302	GPIO_251	GPIO_152	303	304	GPIO_252	GPIO_153	305	306	GPIO_253	GPIO_154	307	308	GPIO_254
GPIO_155	309	310	GPIO_156	GPIO_255	311	312	GPIO_256	GPIO_157	313	314	GPIO_257	GPIO_158	315	316	GPIO_258	GPIO_159	317	318	GPIO_259
GPIO_160	319	320	GPIO_161	GPIO_260	321	322	GPIO_261	GPIO_162	323	324	GPIO_262	GPIO_163	325	326	GPIO_263	GPIO_164	327	328	GPIO_264
GPIO_165	329	330	GPIO_166	GPIO_265	331	332	GPIO_266	GPIO_167	333	334	GPIO_267	GPIO_168	335	336	GPIO_268	GPIO_169	337	338	GPIO_269
GPIO_170	339	340	GPIO_171	GPIO_270	341	342	GPIO_271	GPIO_172	343	344	GPIO_272	GPIO_173	345	346	GPIO_273	GPIO_174	347	348	GPIO_274
GPIO_175	349	350	GPIO_176	GPIO_275	351	352	GPIO_276	GPIO_177	353	354	GPIO_277	GPIO_178	355	356	GPIO_278	GPIO_179	357	358	GPIO_279
GPIO_180	359	360	GPIO_181	GPIO_280	361	362	GPIO_281	GPIO_182	363	364	GPIO_282	GPIO_183	365	366	GPIO_283	GPIO_184	367	368	GPIO_284
GPIO_185	369	370	GPIO_186	GPIO_285	371	372	GPIO_286	GPIO_187	373	374	GPIO_287	GPIO_188	375	376	GPIO_288	GPIO_189	377	378	GPIO_289
GPIO_190	379	380	GPIO_191	GPIO_290	381	382	GPIO_291	GPIO_192	383	384	GPIO_292	GPIO_193	385	386	GPIO_293	GPIO_194	387	388	GPIO_294
GPIO_195	389	390	GPIO_196	GPIO_295	391	392	GPIO_296	GPIO_197	393	394	GPIO_297	GPIO_198	395	396	GPIO_298	GPIO_199	397	398	GPIO_299
GPIO_200	399	400	GPIO_201	GPIO_300	401	402	GPIO_301	GPIO_202	403	404	GPIO_302	GPIO_203	405	406	GPIO_303	GPIO_204	407	408	GPIO_304
GPIO_205	409	410	GPIO_206	GPIO_305	411	412	GPIO_306	GPIO_207	413	414	GPIO_307	GPIO_208	415	416	GPIO_308	GPIO_209	417	418	GPIO_309
GPIO_210	419	420	GPIO_211	GPIO_310	421	422	GPIO_311	GPIO_212	423	424	GPIO_312	GPIO_213	425	426	GPIO_313	GPIO_214	427	428	GPIO_314
GPIO_215	429	430	GPIO_216	GPIO_315	431	432	GPIO_316	GPIO_217	433	434	GPIO_317	GPIO_218	435	436	GPIO_318	GPIO_219	437	438	GPIO_319
GPIO_220	439	440	GPIO_221	GPIO_320	441	442	GPIO_321	GPIO_222	443	444	GPIO_322	GPIO_223	445	446	GPIO_323	GPIO_224	447	448	GPIO_324
GPIO_225	449	450	GPIO_226	GPIO_325	451	452	GPIO_326	GPIO_227	453	454	GPIO_327	GPIO_228	455	456	GPIO_328	GPIO_229	457	458	GPIO_329
GPIO_230	459	460	GPIO_231	GPIO_330	461	462	GPIO_331	GPIO_232	463	464	GPIO_332	GPIO_233	465	466	GPIO_333	GPIO_234	467	468	GPIO_334
GPIO_235	469	470	GPIO_236	GPIO_335	471	472	GPIO_336	GPIO_237	473	474	GPIO_337	GPIO_238	475	476	GPIO_338	GPIO_239	477	478	GPIO_339
GPIO_240	479	480	GPIO_241	GPIO_340	481	482	GPIO_341	GPIO_242	483	484	GPIO_342	GPIO_243	485	486	GPIO_343	GPIO_244	487	488	GPIO_344
GPIO_245	489	490	GPIO_246	GPIO_345	491	492	GPIO_346	GPIO_247	493	494	GPIO_347	GPIO_248	495	496	GPIO_348	GPIO_249	497	498	GPIO_349
GPIO_250	499	500	GPIO_251	GPIO_350	501	502	GPIO_351	GPIO_252	503	504	GPIO_352	GPIO_253	505	506	GPIO_353	GPIO_254	507	508	GPIO_354
GPIO_255	509	510	GPIO_256	GPIO_355	511	512	GPIO_356	GPIO_257	513	514	GPIO_357	GPIO_258	515	516	GPIO_358	GPIO_259	517	518	GPIO_359
GPIO_260	519	520	GPIO_261	GPIO_360	521	522	GPIO_361	GPIO_262	523	524	GPIO_362	GPIO_263	525	526	GPIO_363	GPIO_264	527	528	GPIO_364
GPIO_265	529	530	GPIO_266	GPIO_365	531	532	GPIO_366	GPIO_267	533	534	GPIO_367	GPIO_268	535	536	GPIO_368	GPIO_269	537	538	GPIO_369
GPIO_270	539	540	GPIO_271	GPIO_370	541	542	GPIO_371	GPIO_272	543	544	GPIO_372	GPIO_273	545	546	GPIO_373	GPIO_274	547	548	GPIO_374
GPIO_275	549	550	GPIO_276	GPIO_375	551	552	GPIO_376	GPIO_277	553	554	GPIO_377	GPIO_278	555	556	GPIO_378	GPIO_279	557	558	GPIO_379
GPIO_280	559	560	GPIO_281	GPIO_380	561	562	GPIO_381	GPIO_282	563	564	GPIO_382	GPIO_283	565	566	GPIO_383	GPIO_284	567	568	GPIO_384
GPIO_285	569	570	GPIO_286	GPIO_385	571	572	GPIO_386	GPIO_287	573	574	GPIO_387	GPIO_288	575	576	GPIO_388	GPIO_289	577	578	GPIO_389
GPIO_290	579	580	GPIO_291	GPIO_390	581	582	GPIO_391	GPIO_292	583	584	GPIO_392	GPIO_293	585	586	GPIO_393	GPIO_294	587	588	GPIO_394
GPIO_295	589	590	GPIO_296	GPIO_395	591	592	GPIO_396	GPIO_297	593	594	GPIO_397	GPIO_298	595	596	GPIO_398	GPIO_299	597	598	GPIO_399
GPIO_300	599	600	GPIO_301	GPIO_400	601	602	GPIO_401	GPIO_302	603	604	GPIO_402	GPIO_303	605	606	GPIO_403	GPIO_304	607	608	GPIO_404
GPIO_305	609	610	GPIO_306	GPIO_405	611	612	GPIO_406	GPIO_307	613	614	GPIO_407	GPIO_308	615	616	GPIO_408	GPIO_309	617	618	GPIO_409
GPIO_310	619	620	GPIO_311	GPIO_410	621	622	GPIO_411	GPIO_312	623	624	GPIO_412	GPIO_313	625	626	GPIO_413	GPIO_314	627	628	GPIO_414
GPIO_315	629	630	GPIO_316	GPIO_415	631	632	GPIO_416	GPIO_317	633	634	GPIO_417	GPIO_318	635	636	GPIO_418	GPIO_319	637	638	GPIO_419
GPIO_320	639	640	GPIO_321	GPIO_420	641	642	GPIO_421	GPIO_322	643	644	GPIO_422	GPIO_323	645	646	GPIO_423	GPIO_324	647	648	GPIO_424
GPIO_325	649	650	GPIO_326	GPIO_425	651	652	GPIO_426	GPIO_327	653	654	GPIO_427	GPIO_328	655	656	GPIO_428	GPIO_329	657	658	GPIO_429
GPIO_330	659	660	GPIO_331	GPIO_430	661	662	GPIO_431	GPIO_332	663	664	GPIO_432	GPIO_333	665	666	GPIO_433	GPIO_334	667	668	GPIO_434
GPIO_335	669	670	GPIO_336	GPIO_435	671	672	GPIO_436	GPIO_337	673	674	GPIO_437	GPIO_338	675	676					

After many hours of debugging, we finding issues in NPU_wrapper, the most recent configuration of UART is:

- UART can send 8 bits / 1 byte
- GPIO_0[0] / PIN_AC18 has been assigned to UART RX and GPIO_0[1] / PIN_Y17 has been assigned to UART TX
- Pressing KEY[1] will make the FPGA send data through UART
- The data sent through is the clock cycles it took for the Systolic Array to do its computations
- The FPGA will send this and the Pico will calculate the time taken by multiplying the clock cycles by 1/50MHz.

Results:

```
Command Prompt
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>
C:\Users\iamkr\Documents\part-4-project\Final\demo>python demo.py
Average Duration for tile 0: 5678.40 ns over 1000 runs, where M = 0, N = 8, K = 8
Average Duration for tile 1: 5395.10 ns over 1000 runs, where M = 1, N = 8, K = 8
Average Duration for tile 2: 5577.90 ns over 1000 runs, where M = 2, N = 8, K = 8
Average Duration for tile 3: 5679.50 ns over 1000 runs, where M = 3, N = 8, K = 8
Average Duration for tile 4: 5877.50 ns over 1000 runs, where M = 4, N = 8, K = 8
Average Duration for tile 5: 6166.60 ns over 1000 runs, where M = 5, N = 8, K = 8
Average Duration for tile 6: 6129.80 ns over 1000 runs, where M = 6, N = 8, K = 8
Average Duration for tile 7: 6152.30 ns over 1000 runs, where M = 7, N = 8, K = 8
Average Duration for tile 8: 6295.50 ns over 1000 runs, where M = 8, N = 8, K = 8
C:\Users\iamkr\Documents\part-4-project\Final\demo>
```

```
charan@sandevistan:~/Downloads/demo$ python3 demo.py
Average Duration for tile 0: 1714.82 ns over 1000 runs, where M = 0, N = 8, K = 8
Average Duration for tile 1: 1720.29 ns over 1000 runs, where M = 1, N = 8, K = 8
Average Duration for tile 2: 1752.57 ns over 1000 runs, where M = 2, N = 8, K = 8
Average Duration for tile 3: 1818.95 ns over 1000 runs, where M = 3, N = 8, K = 8
Average Duration for tile 4: 1834.57 ns over 1000 runs, where M = 4, N = 8, K = 8
Average Duration for tile 5: 1904.39 ns over 1000 runs, where M = 5, N = 8, K = 8
Average Duration for tile 6: 1922.63 ns over 1000 runs, where M = 6, N = 8, K = 8
Average Duration for tile 7: 1936.08 ns over 1000 runs, where M = 7, N = 8, K = 8
Average Duration for tile 8: 1967.15 ns over 1000 runs, where M = 8, N = 8, K = 8
charan@sandevistan:~/Downloads/demo$
```

Weekly Logbook Entry Template

Week X

XX/XX/25 – XX/XX/25:

XX X XXX 2025

Time: XX

Title:

Objective:

- **Point 1**

XX X XXX 2025

Time: XX

Title:

Objective:

- **Point 1**